FlashNS: Operator- and Adjoint-Aware Contracted Derivative Execution for High-Order PINNs

Dezhen Lu
University of Tübingen
dezhen.lu@student.uni-tuebingen.de

Updated research draft: October 14, 2026

Abstract

Training a physics-informed network on a high-order objective needs coordinate derivatives of the network and the complete parameter gradient of a loss built from them. FlashNS propagates only the derivative state that a fixed residual consumer reads, and transposes every local dependency of that state into the parameter gradient. For the gradient-enhanced two-dimensional Navier–Stokes objective, an eight-channel state (GL8) replaces the ten-channel third-order jet; its transpose requires the fourth activation derivative. On an NVIDIA H20 in FP64, all 72 planned solves pass unchanged gates, and GL8 reaches them 1.66× faster than a compiled full-jet control and 1.73–3.37× faster than matched adaptations of collapsed Taylor mode, explicit differentiation and exact STDE. On an H100 the same campaign passes all 78 solves, but a fixed L-BFGS cost per evaluation compresses these ratios to 1.22× and 1.29–1.69×. That cost is the optimizer's execution: a registered GH200 study that replaces only the two-loop recursion of L-BFGS by a device-side version leaves all 36 solves bitwise unchanged and shortens optimization 2.21× at 2,048 points (1.82× and 1.48× at 16,384 and 65,536; its timing flag failed on one 5.7% drift repeat). On the transient three-dimensional Ethier–Steinman benchmark, a 17-channel contract passes all gates in 18 of 18 H100 solves; it is 3.60× and 6.96× faster than internal baselines; with all methods compiled, its solves are 1.68× and 1.13× faster than nested autograd and collapsed Taylor mode, which optimizes 1.05× faster. A 36-channel curl-potential contract gives 1.32× objective-gradient throughput. Its L-BFGS training fails a stratified pressure test. With a dense self-scaled Broyden optimizer, all five new seeds of a second registered cohort on GH200 pass every test and an independent audit (median 5,074 seconds to verified accuracy). In five paired seeds, the analytic engine reaches a verified training loss 1.26× sooner than parameter AD and 3.92× sooner than `torch.func`, whose graph replay it evaluates 3.7× faster while equalling it in eager mode.

1 Introduction

Training a physics-informed neural network (PINN) requires derivatives of a network output with respect to coordinates and derivatives of the resulting loss with respect to every network parameter. These are different consumers. For gradient-enhanced incompressible Navier–Stokes training, the residual uses second coordinate derivatives and its gradient uses third derivatives. A fast residual evaluation does not establish a fast training step, and a fast training step does not establish a shorter solve to a prescribed error.

Taylor-mode and forward-propagated derivative methods already address the cost of nested coordinate automatic differentiation (AD) [6, 7, 8]. Residual-specific partial jets, collapsed Taylor propagation and stochastic Taylor contractions are especially relevant to this problem [10, 11, 12].

We build on that setting. Our question is whether a concrete operator contraction, its complete parameter transpose and a practical bounded working-state schedule can shorten an unchanged third-order training solve beyond a compiled full-jet implementation and public derivative methods.

For this objective, independent third partials are not needed: the diffusion consumer asks for the gradient of the Laplacian. We retain the scalar value, coordinate gradient, symmetric Hessian and gradient of the Laplacian. In two dimensions this is an eight-channel state, GL8, instead of the ten channels of a complete third-order state, Q10. Closure through an activation includes products between lower derivatives. Its parameter transpose additionally depends on the fourth activation derivative. Omitting that dependence would change the parameter gradient even if some forward residuals agreed.

Contributions.

1. *Consumer contracts.* We state closed derivative states for two fixed consumers: GL8 for the two-dimensional gradient-enhanced Navier–Stokes residual, C17 for a transient three-dimensional velocity–pressure residual and its spatial gradient, and C36 for a transient three-dimensional curl-potential momentum residual and its gradient. Each contract is closed under affine layers and smooth pointwise activations, and the runtime rejects observables the contract cannot answer.

2. *Complete parameter transposes.* We derive and implement the reverse rules for these states, including the fourth activation derivative that the forward state never stores, the transpose of packed symmetric storage, and a transpose of the stable coefficient program that is actually executed in saturated regions. On H100 the explicit two-dimensional transpose is faster than generic autograd on the same state only from 16,384 points on (Section 5.6).

3. *Execution.* Library GEMMs, explicit pointwise derivative kernels and CUDA Graph replay of the complete loss/gradient program are combined with value-only boundary rows and a chunked mode that accumulates the globally weighted gradient in a bounded workspace. On GH200 a device-side L-BFGS two-loop leaves solves bitwise unchanged and halves the fixed optimizer cost (Section 5.9).

4. *A solve-level comparison protocol.* Every method trains from identical data and initial parameter bytes under one optimizer program and unchanged accuracy gates, with an independent scalar-AD final check, pinned public sources and every attempt retained. Ratios are reported only for jointly accepted pairs. We apply it to eight baselines in two dimensions and to two internal baselines on a three-dimensional benchmark, and report the C36 extension, including its failed acceptance under L-BFGS, and two prospective time-to-accuracy cohorts with a dense quasi-Newton optimizer (one not qualified because of its auditor's tolerance, one that passes 5 of 5) and a paired engine cohort with physically verified solves.

The paper does not claim the first residual-aware derivative method, a globally minimal state, or superiority across PDEs, precisions or GPU families. Representation and execution are separated for one problem on one device (Section 5.6). The two-dimensional H20 campaign, the C36 study (H100, GH200 and CPU) and the Ethier–Steinman solves (H100) use different models, data, seeds, gates and hardware; their timings are never pooled. Table 1 defines the names used throughout.

Table 1: Terminology. Channel counts are per hidden scalar. "Raw" states store unnormalized partial derivatives; GL8 and C36 channels are not Taylor coefficients.

Name	Meaning
Q10	Complete two-dimensional third-order state: value, 2 first, 3 second and 4 third partials (10 channels).
GL8	Two-dimensional consumer state $(z, \nabla z, \nabla^2 z, \nabla\Delta z)$ with packed symmetric Hessian (8 channels).
chunk-512	GL8 executed in 512-point interior chunks that accumulate the globally weighted loss and gradient.
C17	Three-dimensional transient velocity–pressure consumer state: value, spatial gradient, packed spatial Hessian, gradient of the spatial Laplacian, time derivative and its spatial gradient (17 channels).
A24 / Naive Q35	Ethier–Steinman baselines: complete spatial third-order jet plus time channels (24), and all 35 monomials of (x, y, z, t) through degree three; both with autograd parameter gradients.
Nested AD / collapsed jet	External Ethier–Steinman baselines: nested scalar-coordinate PyTorch autograd, and the public collapsed Taylor-mode package `jet`; both with autograd parameter gradients.
C36	Three-dimensional transient consumer state for a curl potential (36 channels, Eq. (17)).
Explicit / analytic transpose	Hand-written reverse rule for a derivative state, as opposed to framework autograd.
Parameter AD	Internal control that executes the same derivative-state forward and obtains the parameter gradient with PyTorch autograd.
Native coordinate AD	Nested framework autograd over coordinates; used as an independent derivative oracle.
Replay	One CUDA Graph execution of the complete loss and all parameter gradients.
Solve	Training from a fixed initialization until both validators accept, or until the frozen budget ends.

2 Related work

Physics-informed networks put PDE constraints into a training objective [1]; gradient-enhanced training also penalizes residual derivatives [2]. NSFnets apply PINNs to the incompressible Navier–Stokes equations in velocity–pressure and vorticity–velocity forms, including Kovasznay flow and the three-dimensional Beltrami (Ethier–Steinman) flow [3]. Our contribution concerns execution of an unchanged derivative-intensive objective rather than a new loss or architecture.

Univariate Taylor propagation and its higher-order reverse sweeps are classical algorithmic differentiation [6]. Taylor-mode AD in JAX avoids the redundant work of nesting first-order AD [7]. Forward Laplacian propagates value, gradient and Laplacian forward through a network to evaluate a second-order operator for neural variational Monte Carlo, with large speedups over nested differentiation [8]. GL8 applies the same forward-propagation principle to a third-order consumer. It must additionally retain the full symmetric Hessian, because nonlinear convection gradients read it, and it supplies a hand-written complete parameter transpose.

Cao et al. [10] already exploit residual-dependent partial jets and specialized layer rules, checking residuals and parameter gradients. FlashNS therefore does not claim priority for propagating only PDE-relevant derivative information. Dangel et al. [11] collapse derivative sums within Taylor-mode computation for linear differential operators, closely related to retaining contractions instead of a full tensor. STDE and its extension [13, 12] use high-order Taylor directions and contractions

to avoid materializing large derivative tensors; the JMLR code link currently points to the public STDE repository [14]. We evaluate the public implementations of these three methods, adapted to the same objective, rather than comparing only against our own compiled full-jet backend.

The three-dimensional model learns a vector potential and takes its curl, so the velocity is divergence-free by construction. Neural Conservation Laws parameterize divergence-free fields with differential forms and prove universality of that construction [9]. We use the classical vector-potential form only as a model choice; our contribution there is the derivative contract that its curl and transient momentum gradient require.

3 Contracted derivative execution

3.1 A fixed scientific objective

We study steady incompressible flow with constant viscosity ν and physical Cartesian coordinates $x = (x_1, x_2)$. A neural network predicts (u, v, p). Its residual is

$$r_u = uu_{x_1} + vu_{x_2} + p_{x_1} - \nu\Delta u, \tag{1}$$
$$r_v = uv_{x_1} + vv_{x_2} + p_{x_2} - \nu\Delta v, \tag{2}$$
$$r_d = u_{x_1} + v_{x_2}. \tag{3}$$

Following the gradient-enhanced objective [2], we retain both the residual and its coordinate gradient:

$$L = \tfrac{1}{2}\,\mathrm{mean}_\Omega\left(\|r\|_2^2 + 0.2\|\nabla r\|_F^2\right) + 5\left(\mathrm{mean}_{\partial\Omega}\,\|(u,v) - (u_*, v_*)\|_2^2 + \mathrm{mean}_{x_1=1}(p - p_*)^2\right). \tag{4}$$

This objective needs third coordinate derivatives. Removing the gradient penalty would change the scientific problem. All implementations use the existing point weights and validation definitions; in particular, the boundary component RMS is not the training vector-square mean.

3.2 A closed representation

For a scalar intermediate z, define

$$S(z) = (z, g, H, t), \qquad g = \nabla z, \quad H = \nabla^2 z, \quad t = \nabla\Delta z. \tag{5}$$

Only independent symmetric Hessian entries are stored. In two dimensions the raw, unnormalized state is

$$(z, z_{x_1}, z_{x_2}, z_{x_1x_1}, z_{x_1x_2}, z_{x_2x_2}, z_{x_1x_1x_1} + z_{x_1x_2x_2}, z_{x_1x_1x_2} + z_{x_2x_2x_2}). \tag{6}$$

This is GL8, compared with the ten channels of Q10. The raw-to-Taylor projection includes the appropriate factorials; GL8 channels must not be read as Taylor coefficients.

For a pointwise activation $y = \phi(z)$, let $a_k = \phi^{(k)}(z)$. Direct application of the chain rule gives

$$g_y = a_1 g, \tag{7}$$
$$H_y = a_1 H + a_2 g g^\top, \tag{8}$$
$$t_y = a_1 t + a_2\,(g\ \mathrm{tr}\,H + 2Hg) + a_3 g\|g\|_2^2. \tag{9}$$

An affine layer applies the same weight matrix to each state channel, with the bias added only to the value channel. Consequently, affine layers and smooth pointwise activations form a closed execution rule for S. The state supplies every quantity in r and ∇r: convection gradients need the Hessian, while diffusion gradients need t. Figure 1 summarizes the forward closure and its transpose.

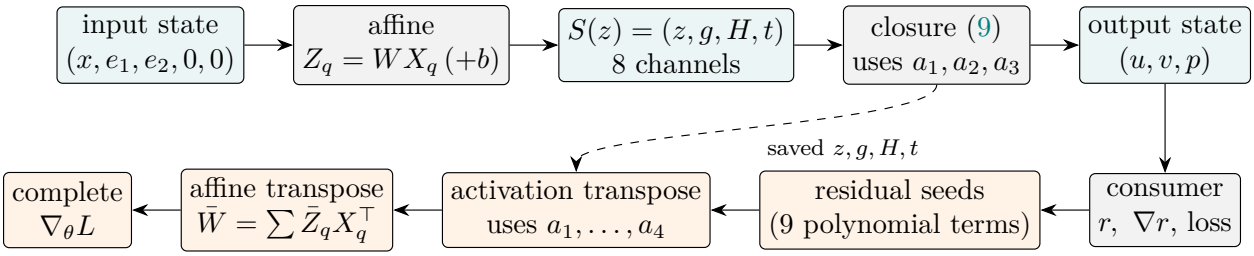


Figure 1: Forward closure (top) and complete reverse transpose (bottom) for GL8; two hidden layers repeat the affine/closure pair. Each layer runs one library GEMM over all point/channel rows, the pointwise closure and its transpose run in native kernels, and the reverse affine reductions reuse flattened GEMMs. Boundary rows carry values only. The complete program is captured once and replayed with live parameter storage. C36 follows the same pattern with 36 channels per scalar, packed symmetric second and third spatial derivatives, and time-mixed channels.

Closure is sufficient for this consumer, not a claim of global minimality or a reconstruction of all third derivatives. For example, $z = x_1^3 - 3x_1x_2^2$ has $S(z) = 0$ at the origin but $z_{x_1x_1x_1} = 6$. A request for an independent third partial cannot be answered from GL8. The runtime rejects unsupported coordinate identities or contractions instead of returning such an observable.

3.3 Complete parameter adjoints

Reducing the forward state alone is insufficient for training. FlashNS transposes the same local closed rules, including their dependence on the primal value. To make that dependence explicit, define $Q = g\,\mathrm{tr}\,H + 2Hg$ and $c = g^\top g$. The differential of the third contracted component is

$$\begin{aligned}\delta t_y = {} & a_1\delta t + a_2\left(\delta g\ \mathrm{tr}\,H + g\ \mathrm{tr}\,\delta H + 2\delta H g + 2H\delta g\right) \\ & + a_3\left(c\,\delta g + 2g(g^\top\delta g)\right) + \left(a_2 t + a_3 Q + a_4 g c\right)\delta z. \end{aligned} \tag{10}$$

Thus the local reverse rule requires a_4, although the forward coordinate state stops at order three. The value, gradient and Hessian differentials are transposed together with (10). For the packed symmetric Hessian, off-diagonal entries use the transpose of symmetric unpacking: the two matrix contributions are added, not treated as two independent stored channels. Appendix A gives the complete local transpose.

If X is a layer input state and $\bar{Z}$ its preactivation adjoint, the complete parameter updates are

$$\bar{W} = \sum_{n,q} \bar{Z}_{nq} X_{nq}^\top, \qquad \bar{b} = \sum_n \bar{Z}_{n,0}, \qquad \bar{X}_{nq} = W^\top \bar{Z}_{nq}. \tag{11}$$

The implementation uses flattened point/channel matrix multiplications for these reductions. A fixed polynomial residual program seeds all nine residual and residual-gradient terms. It is a specific, verified consumer rule rather than a general symbolic residual compiler.

In exact arithmetic, projection of a complete coordinate jet commutes with the closed affine and activation rules. Composing with a loss that depends only on that projection therefore preserves its parameter derivative by the ordinary chain rule. Our numerical checks test both the projected forward quantities and the entire flattened parameter gradient; forward agreement alone would not establish this property.

3.4 Stable activation and bounded working state

The production kernels reuse the existing stable tanh convention. In particular, the first derivative is evaluated with a stable exponential tail instead of relying exclusively on subtraction from a rounded saturated value. Higher derivatives and their reverse rules use the same primal convention. Affine work remains in library GEMMs; pointwise activation and residual transposes use explicit CUDA kernels. CUDA Graph replay reads the current parameter storage rather than a snapshot, which is checked by consecutive in-place updates.

Dirichlet and outlet consumers that need only values use a separate value-only path. At 2,048 interior and 512 boundary points this reduces logical point/channel rows from 25,600 for full Q10 rows everywhere to 20,992 for interior-only Q10 and 16,896 for interior-only GL8; these are workload counts, not measured speedups. The chunked implementation processes interior points in blocks of 512 and accumulates the complete parameter gradient using the original global weights. It never normalizes each chunk separately. Its working-state bound is proportional to chunk size, while fixed data, parameters and optimizer storage remain outside that bound (Appendix B). Floating reduction order can alter line-search trajectories; every backend must still satisfy the same independent final criteria.

The measured GL8 production scope is two-dimensional, constant-viscosity, pointwise affine and tanh networks with first parameter VJPs. The current contract does not support a learned contraction, nonlinear coordinate changes, sample-coupled layers, nonlocal losses, HVPs or double backward.

4 Two-dimensional experimental protocol

4.1 Problem, data and acceptance

The analytic Kovasznay solution is

$$u_* = 1 - e^{\lambda x_1}\cos(2\pi x_2), \quad v_* = \frac{\lambda}{2\pi}e^{\lambda x_1}\sin(2\pi x_2), \quad p_* = \frac{1-e^{2\lambda x_1}}{2}, \qquad \lambda = \frac{1}{2\nu} - \sqrt{\frac{1}{4\nu^2} + 4\pi^2}. \tag{12}$$

We set $\nu = 0.07$ on $[-0.5, 1] \times [-0.5, 1.5]$. The network has widths 2–64–64–3, two tanh hidden layers, a linear head and 4,547 parameters. Training uses 2,048 interior points and 128 points on each of four edges. The 128 pressure-outlet points share the last edge's coordinates; there are 512 boundary coordinates, not 640. We retain the original globally normalized interior, velocity and outlet weights.

The fixed stop-check set contains 8,192 interior points and 257 points on each boundary edge. A separate scalar nested-AD interior set verifies every candidate final solution. Both validations must meet all six limits in Table 2, with finite values. PDE RMS combines the three residuals by the original sum-of-squares convention. Boundary velocity RMS averages velocity components, whereas the training loss uses their vector square. We preserve these definitions rather than reconstructing them from metric names. Five data/weight SHA256 values are identical across every matched run; initialization affects parameters only.

4.2 Optimizer, stopping and finite attempts

Every formal arm uses the same optimizer and stopping program. Adam permits 4,000 updates with learning rate 10^{-3}, halved every 1,000 updates, $\beta = (0.9, 0.999)$ and $\epsilon = 10^{-8}$. Stop checks occur before training and every 100 Adam updates. Strong-Wolfe L-BFGS then permits at most

Table 2: Unchanged numerical acceptance limits. A run succeeds only if both the fixed validation and independent scalar-AD validation satisfy every row.

Metric	Upper limit
Combined PDE residual RMS	0.02
Boundary velocity component RMS	0.01
Outlet pressure RMS	0.01
Relative L_2 error of u	0.01
Relative L_2 error of v	0.02
Relative L_2 error of p	0.02

500 blocks, each with 20 iterations, nominal 25 evaluations, history size 50, learning rate 1, gradient tolerance 10^{-12} and change tolerance 10^{-15}. Stop checks follow each block. Line searches may exceed a nominal evaluation limit; reported closure counts are actual counts and include the Adam evaluations.

The two-dimensional formal seeds are 862101–862106. Development seeds 361901 and 361902 and older regression seeds are excluded from the formal comparisons. No seed, threshold or optimizer budget is changed after an observed result. Every attempt is retained. Nonconvergence consumes its measured cost and counts against the attempted arm; an unsupported execution variant does not turn into a successful baseline through substitution. Paired time ratios are computed only when both members pass and are accompanied by all outcomes.

The Q10/GL8/chunk campaign balances the six permutations of three arms. The subsequent public comparison has six paper-group arms; three sampled permutations and their reverses balance their order over the same six seeds. NVIDIA baselines use separately admitted windows, so their timings are not interleaved with paper-group controls. Each window runs one GPU process at a time; Appendix C gives the device and window records.

4.3 Baselines and execution modes

Q10 compact propagates the complete two-dimensional third-order Taylor state at interior points and values at boundaries. It uses the existing explicit parameter transpose, full-graph Torch compilation and CUDA Graph replay. It is a strong internal representation control, not the public collapsed Taylor package.

Cao ED calls the pinned author's `ed_u_jet` without changing that function. Its value, first derivatives and diagonal second derivatives assemble the Navier–Stokes residual. Coordinate AD of that residual adds the original gradient-enhancement term; parameter AD supplies the complete gradient. This is a matched-objective adaptation of the public implementation.

Collapsed Taylor calls the public `jet(collapsed=True)` transform with exact coordinate directions. The collapsed second-order output supplies the Laplacian, and residual coordinate AD supplies the additional third-order consumer. Parameter AD is included. We test both the default complete loss/gradient program and its traced, compiled variant.

STDE exact calls the public sparse-STDE operator `hess_diag` with dimension two, no randomized Jacobian and `rand_batch_size=0`. The two coordinate directions are exhausted, so this experiment uses no stochastic derivative estimator. JAX forward residual differentiation and parameter backward preserve the same loss. JAX runs on the GPU in FP64; CPU fallback is rejected. DLPack exchange, copies and synchronization with the common Torch optimizer are included in replay and solve wall time. This experiment does not evaluate every STDE++ mixed-partial

construction or its intended high-dimensional stochastic setting.

Torch AD evaluates nested coordinate AD and complete parameter backward on an ordinary dense MLP. **PhysicsNeMo** [15] uses the public `PhysicsInformer` with symbolic Navier–Stokes equations and its autodiff gradient method, followed by residual-gradient AD and parameter backward. It is the full residual path, not just a standalone gradient component.

cuDNN [16] uses an algebraically equivalent 1×1 convolution MLP with coordinate and parameter AD. A cuDNN operator must appear in the actual profiler trace; a fallback is retained as a failed qualification rather than called a cuDNN result. **cuEquivariance polynomial** [17] reuses the inherited FP64 Q10 segmented-polynomial forward and explicit parameter VJP with the native `uniform_1d` backend. That implementation retains its unmodified all-points Q10 policy, including boundaries. Scalar-irreps `Linear` fused-tensor-product variants are separate diagnostics and cannot replace the polynomial formal arm. Actual native operators are required in the profiler.

For Torch-based public adapters and exact STDE, high-order derivatives run only at the 2,048 interior points and values only at 512 boundary points, matching the Q10/GL8 controls' point policy. Public adapters use `torch.tanh`, while production FlashNS preserves its stable tail convention. Mathematically they represent the same activation; numerical full-gradient equivalence is checked rather than assumed. Pinned source commits are listed in Table 34.

4.4 Qualification, caches and timing scopes

Qualification evaluates the complete loss and all 4,547 parameter gradients against the full-jet reference after each of three consecutive in-place updates. The unchanged numerical tolerance is absolute 2×10^{-12} and relative 2×10^{-11}. It also checks actual native execution where required. Passing default/compiled variants are selected by the smallest development median complete-gradient wall time; the selection map and report hashes are frozen before formal seeds. No failed variant is used.

Qualification warms task-owned Torch, native-extension, Triton and JAX caches. Each formal attempt is a fresh process with new parameters, optimizer state and graphs. Persistent JAX compilation caching is explicitly enabled, with compile threshold zero and entry threshold -1; cache settings and hit/miss logs are retained. The resulting solve experiment is a warm-dependency execution comparison. Initial qualification/build cost is reported separately and is not silently amortized over the six seeds.

Complete-gradient qualification uses five groups of 30 replays after a two-second warm-up and reports the median group wall time. The earlier representation/memory study instead uses five independently ordered processes per shape/arm, each with seven groups of 100 replays; these are separate experiments. Replay timing includes full loss, every parameter gradient, chunking and interoperation costs, but excludes optimizer, stop validation and setup.

Optimization wall time includes training, optimizer work, line searches, synchronization, stop checks and independent final acceptance. Setup plus optimization additionally includes per-run engine preparation. It excludes interpreter startup, common data creation and teardown; full child-process wall time includes those costs. We report paired geometric means of time ratios, not ratios of unpaired medians. Small-sample ranges are descriptive, not a general-PDE confidence guarantee.

All matched experiments use FP64 with TF32 disabled on an NVIDIA H20 (SM90, 97,871 MiB), driver 580.76.05, CUDA toolkit 13.0.88, Torch 2.12.1+cu130, Triton 3.7.1, JAX 0.11.2 and the native cuEquivariance ops package 0.12.0.

Table 3: Development GPU qualification on seed 361902. Median complete loss/all-parameter-gradient wall time from five groups of 30 replays; optimizer, validation and setup excluded. Each passing variant satisfies three successive updated-parameter checks. JAX exchange is included.

Method	Selected execution	Gradient ms	Max gradient error
FlashNS GL8	Native + graph	1.583	7.55×10^{-15}
Q10 compact	Inductor + graph	3.267	3.55×10^{-15}
Collapsed Taylor	Compiled + graph	3.751	5.33×10^{-15}
Cao ED	Compiled + graph	5.725	5.33×10^{-15}
Torch coordinate AD	Compiled + graph	7.651	5.33×10^{-15}
STDE exact (2D)	JAX JIT	9.459	5.33×10^{-15}
PhysicsNeMo	Compiled + graph	6.506	5.33×10^{-15}
cuDNN convolution AD	Compiled + graph	74.004	5.33×10^{-15}
cuEquivariance polynomial	Native polynomial + graph	3.340	5.33×10^{-15}

5 Two-dimensional results

5.1 Complete-gradient qualification

All nine paper-group development variants in Table 3 pass the complete-gradient checks; their largest observed absolute parameter-gradient discrepancy is 7.55×10^{-15}. Selection uses development replay latency only, never observed formal solve speed or convergence. The separate NVIDIA qualification attempts nine variants, of which five pass; failed native-path or complete-gradient checks remain in the raw reports, consume their measured process cost, and leave their formal cells not-run. Qualification process-wall costs, including unsuccessful variants and the original author recipe where applicable, are 137.927 s (NVIDIA group) and 263.687 s (paper group); they are separate from warm-dependency formal solve timings. Appendix C records the cuEquivariance initialization repair and the scalar tensor-product variants that failed higher-order autograd qualification.

5.2 What the reported solve times measure

Across all 54 accepted public solves the largest final v relative error is 0.0199–0.0200 for every method, at its limit of 0.02, while the other five criteria retain margin (Table 6). The relative L_2 error of v is therefore the binding criterion in this problem: a reported solve time is, in practice, the time to the first stop check at which v crosses 0.02 with all other criteria already satisfied. Every accepted solve completes the 4,000-update Adam phase and stops during L-BFGS, since each run records more than 4,000 loss/gradient evaluations.

Stop checks run every 100 Adam updates and after each L-BFGS block of at most 20 iterations and nominally 25 evaluations. The measured time thus overshoots the first true crossing by at most one check interval. For GL8, one nominal block is at most about 0.4% of its 6,261–9,032 recorded evaluations. The same rule applies to every arm, so check granularity enters paired ratios only through differing optimization trajectories.

5.3 Six-seed representation and chunk comparison

Native GL8 is faster than Q10 in all six paired solves. The optimization-only geometric mean is 1.479×, while setup plus optimization is 1.659× (pair range 1.460–1.809). Full-process speedup is 1.635×. Chunk-512 has setup-plus-optimization speedup 1.423×. Its six-seed median is longer than

Table 4: Representation campaign: all 18 attempts pass fixed and independent acceptance. Time columns are medians in seconds. Last column is the geometric mean of six paired Q10/candidate ratios for setup plus optimization, not the ratio of table medians.

Method	Accepted	Optimization	Setup + opt.	Process	Paired speedup
Q10 compact	6/6	33.660	38.519	41.798	1.000×
FlashNS GL8	6/6	22.756	23.110	25.598	1.659×
GL8 chunk-512	6/6	28.004	28.354	31.091	1.423×

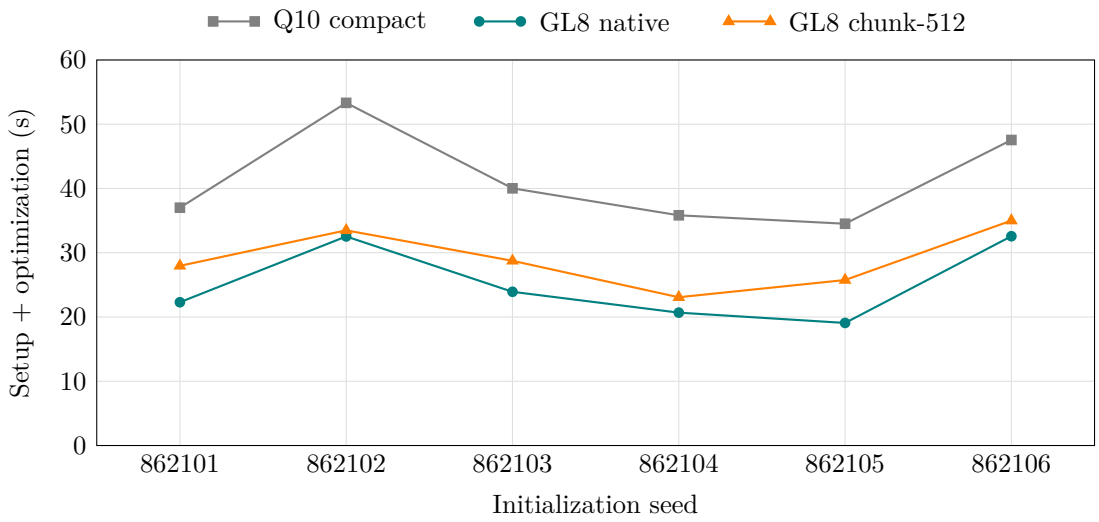


Figure 2: Every observed paired solve in the representation campaign. All 18 pass. Connected points identify backends across different initialization seeds; they do not interpolate a continuous experimental variable.

native GL8, consistent with the extra replay and accumulation schedule. Development preflight cost is 24.962 seconds and is reported separately.

5.4 Source-pinned public solves

The public matrix plans 54 cells and records 54 actual formal child attempts, of which 54 pass every acceptance criterion. The priority paper group records 36 of 36 planned attempts. All observed formal initial parameter hashes match the corresponding seeds in the original representation campaign; the six planned initializations are reused across experiments, not additional independent seeds for each backend.

Paired baseline/GL8 setup-plus-optimization ratios are Collapsed Taylor: 1.730× (6 accepted pairs, range 1.482–1.906); Cao ED: 2.312× (6 accepted pairs, range 2.024–2.607); STDE exact (2D): 3.368× (6 accepted pairs, range 2.833–3.862); Torch coordinate AD: 2.994× (6 accepted pairs, range 2.712–3.259). They describe the matched adaptations and common optimizer pipeline. They are not the original papers' speedup claims.

An unmodified run of the original Cao NS script (its own six-hidden-layer FP32 lid-driven-cavity recipe, three 1,000-Adam repeats) completes on the same GPU with AD and ED mean times of 11.132 and 4.613 seconds (ratio 2.413). This confirms the public script runs; it is not a Kovasznay gPINN solve or a FlashNS comparison, and its timing is not pooled.

Table 5: Public comparison. Outcomes are accepted/nonconverged/runtime-or-correctness-failure/not-run out of six planned seeds. Time medians in seconds retain all attempted solves. Final column is paired baseline/GL8 setup-plus-optimization geometric mean and number of jointly accepted pairs; values above one favor GL8. NVIDIA arms run in separate windows.

Method	Outcomes	Opt. s	Setup + opt. s	Process s	Ratio / n
FlashNS GL8	6/0/0/0	23.131	23.496	25.818	1.000 / 6
Q10 compact	6/0/0/0	34.090	39.248	42.877	1.658 / 6
Collapsed Taylor	6/0/0/0	35.955	40.415	43.775	1.730 / 6
Cao ED	6/0/0/0	50.055	54.810	58.232	2.312 / 6
Torch coordinate AD	6/0/0/0	64.800	70.682	74.447	2.994 / 6
STDE exact (2D)	6/0/0/0	75.376	78.650	81.400	3.368 / 6
PhysicsNeMo	6/0/0/0	54.479	72.506	77.286	3.108 / 6
cuDNN convolution AD	6/0/0/0	523.492	531.241	534.903	22.865 / 6
cuEquivariance polynomial	6/0/0/0	32.155	35.729	38.697	1.555 / 6

Table 6: Largest final error over all accepted seeds and both validators, rounded to four decimals. Acceptance uses the unrounded values and every criterion; n is the number of accepted seeds.

Method	n	PDE	Boundary	Outlet	u	v	p
FlashNS GL8	6	0.0138	0.0093	0.0025	0.0095	0.0200	0.0099
Q10 compact	6	0.0147	0.0096	0.0024	0.0096	0.0199	0.0109
Collapsed Taylor	6	0.0132	0.0096	0.0025	0.0093	0.0200	0.0091
Cao ED	6	0.0136	0.0095	0.0026	0.0091	0.0200	0.0103
Torch coordinate AD	6	0.0146	0.0099	0.0028	0.0097	0.0199	0.0102
STDE exact (2D)	6	0.0141	0.0097	0.0030	0.0094	0.0200	0.0096
PhysicsNeMo	6	0.0146	0.0095	0.0021	0.0096	0.0199	0.0097
cuDNN convolution AD	6	0.0157	0.0100	0.0023	0.0093	0.0200	0.0112
cuEquivariance polynomial	6	0.0150	0.0098	0.0024	0.0094	0.0199	0.0097

5.5 Memory and replay tradeoff

At 16,384 interior points, chunk-512 reduces native GL8 setup/capture allocated peak from 312.79 to 45.88 MiB, a reduction of 85.33%, and ready reserved memory from 340 to 76 MiB. Full-gradient replay increases from 11.511 to 17.908 ms, a 55.58% cost. At 2,048 points, native GL8 remains the speed-oriented option. No maximum trainable scale or device-traffic reduction is inferred from these counters.

5.6 Representation and execution on H100

The H20 comparison combines the derivative state with its execution.

Design. A separately registered H100 study separates the two. It uses the unchanged two-dimensional source at its pinned commit, the same FP64 objective, data and stable activation, and the same benchmark: a fresh process per measurement, a full loss/gradient check against the dense Q10 oracle, a 2-second warmup and 7×100 timed CUDA Graph replays.

Six cells change one factor at a time:

- A: compact Q10 compiled by Inductor;

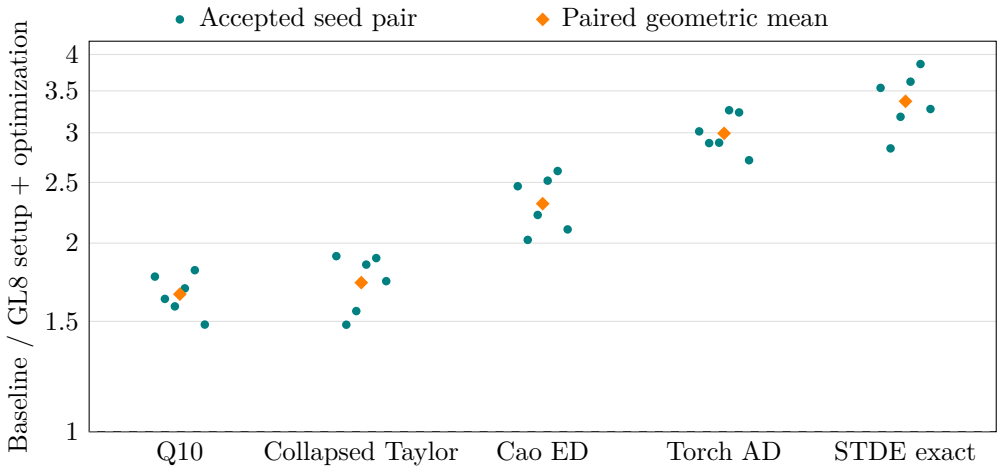


Figure 3: Paper-group solve ratios for every jointly accepted initialization pair, on a logarithmic axis. Above one favors GL8; the dashed line is equal time. Horizontal offsets separate seed observations, not additional experimental conditions. Table 5 retains all attempted outcomes and the accepted-pair denominators.

Table 7: Earlier independent-process replay/memory study: five processes per shape/arm, seven groups of 100 replays per process. Latency is the median of process medians. Allocator measurements are taken before allocating the full-jet correctness oracle; they measure Torch allocation/reservation, not total GPU process memory.

Interior	Method	Gradient ms	Setup peak MiB	Ready reserved MiB
2,048	Q10 compact	3.272	87.16	102.00
2,048	FlashNS GL8	1.578	69.61	76.00
2,048	GL8 chunk-512	2.271	42.92	56.00
16,384	Q10 compact	23.539	456.72	642.00
16,384	FlashNS GL8	11.511	312.79	340.00
16,384	GL8 chunk-512	17.908	45.88	76.00

- B: compact GL8 with the explicit output-state transpose, compiled by Inductor;
- C: the same GL8 layout and transpose with the native pointwise kernels, uncompiled;
- D and E: A and B run eagerly;
- F: the GL8 state with generic parameter autograd, compiled by Inductor.

There is no native Q10 kernel, so the total C/A is decomposed as (B/A)(C/B).

The protocol stated beforehand that an earlier A100 measurement at 2,048 points had found compiled GL8 no faster than compiled Q10. A cell's time is the median over five processes of each process's median. Interior points run from 2,048 to 65,536 with 512 boundary coordinates, and four further topologies are measured.

Validity checks. All 244 tasks complete. The 226 timed processes pass the oracle with gradient errors of at most 8.0×10^{-15}, and the four repeated tasks agree within 0.6%.

The registered A/A control (cell A under a second label) passes at 16,384 points (1.0002) but fails at 2,048 points (1.057 against a 3% gate), so every 2,048-point comparison failed a validity gate. As a post-hoc observation, the processes at 2,048 points fall into two states about 6% apart in every compiled and graph cell, on all four GPUs and at a constant 1,980 MHz SM clock, while each process's own samples vary by at most 0.5%. The cause was not found, and with five bimodal processes the bootstrap intervals at this size are not informative.

Table 8: H100 full loss and all-parameter-gradient replay (2–64–64–3 network). Times are milliseconds per gradient; ratios are the first cell's time over the second's (above one favours the second cell). Representation A/B and execution B/C multiply to the total A/C; F/B is generic autograd over the explicit transpose on the same GL8 state. The 2,048-point row failed the registered A/A gate. Eager cells D/E were measured at 2,048 and 16,384 points only.

Points	A Q10	B GL8	C native	F autograd	A/B	B/C	A/C	F/B
2,048	0.282	0.259	0.165	0.265	1.09	1.57	1.71	1.03
4,096	0.658	0.493	0.278	0.535	1.34	1.77	2.37	1.09
8,192	1.519	1.126	0.614	1.206	1.35	1.83	2.48	1.07
16,384	2.918	1.788	1.172	2.342	1.63	1.53	2.49	1.31
32,768	5.731	3.462	2.212	4.557	1.66	1.56	2.59	1.32
65,536	11.422	6.939	4.338	9.090	1.65	1.60	2.63	1.31

Decomposition. Native execution makes the same GL8 computation 1.53–1.83 times faster at every size (Table 8). The representation effect is small and unresolved at the original 2,048 points (1.09, inside the failed A/A gate), 1.34–1.35 at 4,096–8,192 points, and 1.63–1.66 from 16,384 points on. The explicit transpose shows no effect against generic autograd at 2,048 points (1.03) and is 1.07–1.09 times faster at 4,096–8,192 points and 1.31 times faster from 16,384 points on. Run eagerly, GL8 is 1.27 and 1.91 times faster than Q10 at 2,048 and 16,384 points.

Between 8,192 and 16,384 points the representation and transpose effects jump while the execution effect falls (1.83 to 1.53). This is a change of regime rather than a smooth trend; its cause was not profiled, because profiles exist only at 2,048 and 16,384 points.

Table 9: Kernel time of one untimed replay in microseconds. GEMM is cuBLAS in every cell: Inductor emitted no matrix-multiplication templates. Pointwise is Inductor-generated code for A and B, and native islands plus reductions and other kernels for C.

Points	Cell	Kernels	GEMM	Pointwise	Total
2,048	A Q10	48	87	196	284
2,048	B GL8	41	75	170	245
2,048	C native	26	75	73	152
16,384	A Q10	67	711	2,202	2,915
16,384	B GL8	43	571	1,193	1,766
16,384	C native	26	573	570	1,149

Profiles. The profiles locate both effects (Table 9). B and C spend the same time in GEMMs, so the execution effect lies entirely in the pointwise work: about 1,190 against 570 microseconds at 16,384 points. Q10's GEMM time is 1.25 times GL8's, the ratio of their channel counts, and its

pointwise time 1.84 times. The eager cells launch 842 (GL8) and 1,155 (Q10) kernels per gradient, against 41–67 compiled and 26 native.

Topology. At 16,384 points the decomposition hardly depends on the network. For hidden widths 32 and 128 and for three and four hidden layers of width 64, the representation effect is 1.61–1.66, execution 1.49–1.55 and the total 2.39–2.56. At 2,048 points the representation effect varies from 0.92 for width 32 (GL8 slower) to 1.63 for width 128, and is 1.26 and 1.39 for three and four hidden layers. These 2,048-point cells have three processes each and share the bimodality above.

Solves. Development solves of cells A, B and C on the six formal seeds at 2,048 points all pass acceptance. The registered setup-plus-optimization geometric means are 1.16 for C over A (0.91–1.40), 0.98 for B over A and 1.19 for C over B. Optimization time alone gives 0.95, 0.96 and 0.99; the 1.16 comes from setup. Q10 setup takes 4.7–5.8 seconds against 0.3 seconds for the native path, but these are warm-cache values (see below).

Averaged over a whole solve, each evaluation takes 2.5–3.8 milliseconds of optimization time against 0.17–0.28 milliseconds of gradient replay. A post-hoc split of the progress records locates the difference. The 4,000 Adam updates cost 0.25–0.39 milliseconds per evaluation, close to replay. The L-BFGS phase costs 6.3–7.2 milliseconds per evaluation in every cell (2,115–5,231 evaluations per solve); this phase includes the two-loop recursion, the strong-Wolfe line search and the block-boundary stop checks. The L-BFGS implementation, not the gradient, therefore dominates a solve on this device at this size, and evaluation counts vary from 6,115 to 9,231 between seeds and cells. On H100 at 2,048 points, GL8's 1.71-fold faster gradient does not translate into faster optimization.

Solves at larger point counts. A second registered H100 study, whose protocol was committed before its batch ran, asks whether the solve-level gain returns when the gradient is a larger share of each evaluation. Cells A, B and C solve the problem with 16,384 and 65,536 interior points from the six formal initializations, one fresh process per solve. These are new problem instances with different collocation points; the network, optimizer program, validation sets and acceptance are unchanged. The registered rule declares a solve-level gain at a size when the optimization-only geometric mean exceeds 1.03 and at least five of six jointly accepted pairs favour the faster cell. All 36 solves pass acceptance, with 5,918–8,977 evaluations.

Table 10 gives the result. The rule finds a gain for every pair at both sizes, and all six pairs favour the faster cell each time. In optimization alone, C is 1.48 times faster than A at 16,384 points and 1.88 times faster at 65,536; the protocol expected about 1.4 and 2.0. Per evaluation the ratios are 1.43 and 1.95. The difference between the two measures comes from seed-to-seed variation in evaluation counts. The representation (A/B) contributes 1.26 and 1.41, the execution (C/B) 1.18 and 1.34.

A post-hoc phase split explains the trend. In the Adam phase an evaluation costs only 0.05–0.12 milliseconds more than the replay median for the same cell and size, so the ablation's replay medians carry over to these solves. In the L-BFGS phase an evaluation costs 5.9–7.1 milliseconds more than the replay, at all three sizes and in every cell. This cost lies outside the gradient and does not depend on the point count; it was not profiled further. It is 22–43 times the gradient replay at 2,048 points and 0.5–1.5 times at 65,536 points, where the per-evaluation solve ratio (1.95) approaches the replay ratio (2.63).

The registered setup-plus-optimization ratios (C over A 1.21 at 16,384 points and 2.12 at 65,536) depend on compiler-cache state. Each lane keeps its own Inductor cache. The first solve of a cell and size on a lane took 22–23 seconds of setup for A and 12–14 seconds for B; two B solves started

Table 10: Complete H100 solves at larger point counts (six formal seeds per cell, all accepted). Phase costs are milliseconds per complete evaluation: Adam is the 4,000-update phase, L-BFGS everything after it. Replay is the ablation's registered median for the same cell and size. Pair ratios are paired geometric means (range) of the first cell's time over the second's; "favour" counts jointly accepted pairs favouring the second cell.

Points	Cell	Replay	Adam	L-BFGS	Evaluations
16,384	A	2.918	3.00–3.03	8.89–9.27	5,972–8,924
	B	1.788	1.86–1.88	7.81–8.01	6,437–8,977
	C	1.172	1.25–1.26	7.43–7.83	6,065–8,213
65,536	A	11.422	11.47–11.52	17.48–17.65	5,918–8,573
	B	6.939	7.00–7.04	12.85–13.09	6,341–8,721
	C	4.338	4.43	10.46–10.91	6,260–8,811

Points	Pair	Optimization only	Per evaluation	Favour
16,384	A/C	1.48 (1.31–1.80)	1.43 (1.34–1.57)	6/6
	A/B	1.26 (1.12–1.43)	1.25 (1.21–1.32)	6/6
	B/C	1.18 (1.02–1.28)	1.14 (1.06–1.20)	6/6
65,536	A/C	1.88 (1.75–2.05)	1.95 (1.90–2.01)	6/6
	A/B	1.41 (1.31–1.55)	1.46 (1.42–1.50)	6/6
	B/C	1.34 (1.24–1.46)	1.34 (1.31–1.37)	6/6

together at the beginning of the batch took 47 seconds. Later solves took 4.7–5.9 and 3.3–4.3 seconds. Two native solves started at the beginning of the batch took 65 seconds of setup, the others 0.3–1.4 seconds; the cause was not investigated. The same check on the ablation shows that every 2,048-point Q10 solve ran after earlier tasks of its cell on the same lane, so the 4.7–5.8 seconds quoted above are warm-cache values and the 2,048-point setup comparison is warm against warm. Warm Q10 setup (4.7–5.9 seconds) remains slower than native setup (0.3–1.4 seconds), a real difference between the cells. At 16,384 and 65,536 points, however, queue order decided which seeds paid a cold compile. This widens the per-seed setup-plus-optimization range (0.62–2.14 for C over A at 16,384 points) and lifts the 65,536-point value (2.12) above optimization alone (1.88). The decisions use optimization time alone.

On H100 the solve-level gain is therefore a matter of problem size: it is absent at 2,048 points, where a fixed L-BFGS cost per evaluation dominates, and it is 1.48 and 1.88 at 16,384 and 65,536 points. The H20 gain at 2,048 points (Section 5.3, 1.48 in optimization alone) is consistent with this picture, since a compiled Q10 gradient took 3.27 milliseconds there. The H20 L-BFGS cost was not measured separately, the problem instances differ, and no H100 ratio is pooled with the H20 campaign. All of this is one problem and one optimizer implementation.

5.7 Reduced precision on H100

Many PINN codes train in FP32. A registered H100 study, whose protocol was committed before its batch ran, asks whether this objective needs FP64, and how the precision changes the speed of compiled Q10 (cell A) and native GL8 (cell C).

- **Arms.** Three precisions: FP64, FP32, and FP32 with TF32 matrix products.
- **Source change.** The pinned source accepted only FP64, so a 28-line patch relaxes only dtype checks in eight modules. It leaves FP64 gradients bit-identical, and the native kernels use

Table 11: Reduced precision on H100. Replay is milliseconds per full gradient (median of five processes). Error is the relative L_2 gradient error against the FP64 dense-Q10 oracle. Solves: accepted out of six by the FP64 check, and the paired FP64/reduced optimization-only geometric mean (range); above one favours the reduced precision.

Cell	Precision	2,048	16,384	65,536	Error	Solves (2,048 / 65,536)
A	FP64	0.281	2.920	11.440	–	6/6 / 6/6
	FP32	0.183	1.573	6.004	$1.4\text{–}2.7 \times 10^{-7}$	6/6, 0.98 / 6/6, **1.53**
	TF32	0.187	1.534	5.829	1.5×10^{-4}	0/6 / 0/6
C	FP64	0.164	1.165	4.343	–	6/6 / 6/6
	FP32	0.136	0.662	2.399	$1.5\text{–}2.7 \times 10^{-7}$	6/6, 0.97 / 6/6, **1.36**
	TF32	0.117	0.640	2.282	$1.5\text{–}2.2 \times 10^{-4}$	0/6 / 0/6

their existing FP32 path.

- **Measurements.** Replays at 2,048, 16,384 and 65,536 points, and complete solves at 2,048 and 65,536 points on the six formal seeds.
- **Acceptance.** Data and parameters are cast from the FP64 originals. Every stop check and the final acceptance evaluate the parameters cast back to FP64 on the FP64 problem, with unchanged thresholds.
- **Registered rule.** A reduced precision is sufficient when 6/6 solves pass, and faster when its optimization-only geometric mean exceeds 1.03 with at least five of six favourable pairs.

All 162 tasks ran.

FP32 suffices for this objective at these gates. Its gradients differ from the FP64 oracle by $1.4\text{–}2.7 \times 10^{-7}$ in relative L_2, and all 24 FP32 solves pass the FP64 acceptance in both cells and at both sizes (Table 11). The stop rule ends every FP32 and FP64 solve at the gates, with its worst metric at 0.95–1.00 of the limit, so the study cannot show whether FP32 would reach tighter accuracy. TF32 does not: its gradient error is $1.5\text{–}2.2 \times 10^{-4}$, and all 24 TF32 solves fail. A post-hoc look at these solves shows that L-BFGS makes only 500–850 iterations over its roughly 500 blocks, about one per block. The final metrics stay 10–22 times above their limits, with nearly every gate failing.

In replay, FP32 makes cell A 1.54, 1.86 and 1.91 times faster than FP64 at the three sizes, and cell C 1.21, 1.76 and 1.81 times. Against FP32, TF32 is 2.5–5% faster at 16,384 and 65,536 points; at 2,048 points it is 2% slower for A and 16% faster for C. In whole solves the registered rule finds no gain at 2,048 points (0.98 and 0.97), where the fixed L-BFGS cost dominates. At 65,536 points FP32 shortens optimization by 1.53 (A) and 1.36 (C), with all six pairs favourable. Native GL8 keeps its advantage in FP32: C over A is 1.67 in optimization alone at 65,536 points (1.88 in FP64) and no gain at 2,048 points, as in FP64.

5.8 The H20 campaign on H100

The review also asked for the H20 protocol on H100. A registered batch runs the two-dimensional campaign unchanged on one four-GPU H100 node: the same 460 source, adapter and snapshot files, frozen data, six formal initializations, optimizer budget and acceptance. Its plan was prepared earlier but never ran; only the batch's job name changed. Each backend first qualifies its default and compiled variants against the dense Q10 oracle, and the faster qualified variant runs the solves.

Table 12: The H20 campaign on H100 (2,048 points, six formal seeds, all accepted). Ratios are paired baseline/GL8 geometric means (range) over six pairs; above one favours GL8. Setup + opt. is the campaign's registered statistic, Optimization only is supplementary. The H20 column repeats Tables 4 and 5.

Baseline	H100 median s	H100 setup + opt.	H100 opt. only	H20 setup + opt.
GL8 (papers lane)	20.8	1.000	1.000	1.000
Q10 compact	25.3	1.22 (1.12–1.39)	1.02	1.66
Collapsed Taylor	28.6	1.37 (1.28–1.63)	1.21	1.73
Cao ED	28.0	1.29 (1.13–1.42)	1.09	2.31
Torch coordinate AD	39.4	1.90 (1.79–2.00)	1.67	2.99
STDE exact (2D)	37.2	1.69 (1.45–1.88)	1.58	3.37
PhysicsNeMo	44.1	1.93 (1.51–2.43)	1.52	3.11
cuEquivariance polynomial	32.5	1.49 (1.29–1.94)	1.34	1.56
cuDNN convolution AD	660.0	30.1 (25.9–35.7)	30.4	22.9
GL8 chunk-512	22.3	1.10 (1.04–1.31)	1.10	1.42

GL8 runs on every lane, so every pair shares a seed and a GPU. The analysis is the campaign's own audit, with a supplementary reader registered before the run.

All 78 solves pass and are rechecked by complete CPU scalar AD; the qualification gradients stay at most 5.5×10^{-4} of their budget. Compilation won for ED, collapsed Taylor, torch AD, PhysicsNeMo and cuDNN; for example, it made the collapsed-Taylor replay 3.2 times faster (2.36 to 0.75 ms). The cuEquivariance native arm failed qualification on H100, because its fused tensor product has no registered double-backward formula; its polynomial arm runs. Every solve started with a warm compiler cache.

On H100, GL8 still reaches the gates faster than every baseline (Table 12), but by much less than on H20. Against compiled Q10 the registered ratio falls from 1.66 to 1.22, against collapsed Taylor from 1.73 to 1.37, and against ED, torch AD and STDE from 2.31–3.37 to 1.29–1.90. In optimization alone, Q10 (1.02) and ED (1.09) are nearly level with GL8; the remaining Q10 gap is setup, 4.8 s with a warm compiler cache against 0.3–0.6 s native.

The supplementary phase split shows the mechanism of Section 5.6. In the Adam phase, an evaluation costs 0.24–0.25 ms for GL8, 0.37–0.40 for Q10, 0.86–0.89 for collapsed Taylor and 2.0–2.3 for STDE and torch AD. In the L-BFGS phase, every backend except cuDNN costs 6.3–9.0 ms per evaluation. The larger H20 ratios are consistent with a device where the FP64 gradient is a larger share of each evaluation; the H20 L-BFGS cost was not measured separately. On H100 at this size the shared optimizer cost compresses them; at larger point counts (Section 5.6) the gain returns. The memory/replay lane reproduces the chunk trade-off at 16,384 points: chunk-512 allocates an 85% lower setup peak than GL8 (45.9 against 312.8 MiB) at 3.4 times its replay time. No H100 ratio is pooled with H20.

5.9 Optimizer execution on GH200

The phase splits above put most of a 2,048-point solve outside the gradient: on H100 the L-BFGS phase costs 5.9–7.1 ms per evaluation in every cell against 0.17–0.28 ms of gradient replay, and post hoc it is 93% of the optimization time. The cause is in the optimizer's execution, not in the problem. `torch.optim.LBFGS` keeps the two-loop coefficients as device tensors and passes each one as `alpha=`, which converts it to a host number: about 109 device synchronizations and 555 small operators per iteration at history 50. A registered GH200 study, whose protocol was committed

Table 13: Optimizer execution on GH200 (native GL8, FP64, six formal seeds per size). Identical: pairs with the same final parameters, evaluation and iteration counts and acceptance. Ratios are paired stock/light geometric means (range) of optimization-only seconds; above one favours light. Phase costs are median milliseconds per evaluation.

Points	Identical	Stock / light	Favour	Adam ms (both)	L-BFGS ms stock / light
2,048	6/6	**2.21** (1.96–2.38)	6/6	0.22	6.64 / 2.93
16,384	6/6	**1.82** (1.68–2.15)	6/6	0.70	6.97 / 3.43
65,536	6/6	**1.48** (1.33–1.59)	6/6	2.24	8.49 / 4.91

before the batch ran, asks whether replacing only the two-loop recursion by a device-side version shortens complete solves *without changing them*.

- **Cells.** *stock*: `torch.optim.LBFGS` as installed (torch 2.14.1). *light*: the installed `step` source with exactly its two-loop block replaced. The coefficients stay on the device; one Triton kernel per history entry computes $\alpha_i = \rho_i\, s_i^\top q$ and the update $q \leftarrow q - \alpha_i y_i$ with a single rounding, which reproduces `add_(alpha=)` bitwise (0 of 909,400 elements differ in a development check; a round-twice form differs in 25%). Once the history is full, the rows are mirrored into a ring buffer and one captured CUDA Graph per ring position replays the whole two-loop. The memory update, the strong-Wolfe line search and the stop tests are the runtime's own code.

- **Solves.** Native GL8 in FP64, the unchanged `solve.py` program and acceptance, at 2,048, 16,384 and 65,536 interior points on the six formal initializations: 36 solves plus four drift repeats, one fresh process each, four lanes on one GH200 node, every timed task starting with a warm native and Triton cache.

- **Registered rule.** Per size, `EXECUTION_GAIN_IDENTICAL` requires all six pairs to be identical (final parameter SHA-256, evaluation and iteration counts, acceptance), a stock/light optimization-only geometric mean above 1.03 and at least five favourable pairs; any non-identical pair blocks the claim. A drift repeat that is not identical or whose time is outside 5% marks the timing `NOT_VALID`; identity results stand.

- **Platform note.** `torch.randn` on the CPU is not bitwise identical between x86-64 and the GH200 node's aarch64 CPU, while `torch.rand` is. The GH200 formal initializations are therefore the aarch64 draws, pinned by hash in the protocol beside the x86 hashes of the H100 studies; the two platforms share the collocation data but not the initial parameters bitwise. A first attempt with the x86 hashes was refused at admission before any GPU work, and a second failed in the prebuild on a wrapper import; neither ran a scored task.

All 36 pairs are identical (Table 13): the light optimizer changes execution only. Optimization is 2.21 (1.96–2.38) times shorter at 2,048 points, 1.82 at 16,384 and 1.48 at 65,536, with all six pairs favourable at every size; the Adam phase is unchanged and the L-BFGS phase falls from 6.6–8.5 to 2.9–4.9 ms per evaluation. By identity, geometric mean and favour counts the registered outcome is `EXECUTION_GAIN_IDENTICAL` at every size, but the registered drift rule marks the timing `NOT_VALID`: the four repeats are identical to their first runs and three are within 5% in time (1.005, 0.976, 0.995), while one stock repeat at 16,384 points ran 5.7% faster the second time (0.943). The variance is on the stock side, whose host-bound L-BFGS phase ran beside three other lanes; the light repeat was within 0.5%. We report the decision with that label; a re-run with one lane per node would be needed to validate the timing under the registered rule, and the ranges above lie

well outside the 5% drift. The 2,048-point ratio (2.21) is below the 2.4–2.9 predicted from three development pairs run one at a time; the other two sizes are inside their predicted ranges.

Two development measurements, not registered results, locate the next costs. A single-program version of the two-loop (one thread block, q and r in registers, one block reduction per history entry) takes 0.147 ms against 0.498 ms for the graph replay, and lowers the optimizer's cost per iteration from 1.27 to 0.92 ms on a synthetic problem; the remaining cost is the line search's host round trips, during which the GPU is idle for most of each evaluation at 2,048 points. Splitting the 65,536-point problem over the four GPUs of one node, with each rank's loss and gradient summed through symmetric buffers in a fixed rank order (17 μs per all-reduce, all ranks bitwise equal, so the replicated optimizer never diverges), shortens one light solve from 21.8 to 11.4 s; at 2,048 points the same split is slower, because every evaluation then waits for the slowest of four hosts.

6 Three-dimensional transient extension: C36

6.1 Scientific problem and derivative contract

The mathematical source is OpenAI Math376's incompressible box-transport construction [20]. We instantiate one compactly supported box motion in the unit cube over $t \in [0,1]$. For prescribed smooth centers $c(t)$, a determinant-one diagonal dilation $D(t)$ and a smooth cutoff χ, the construction gives

$$y = x - c(t), \qquad L = \dot{D}D^{-1}, \qquad \operatorname{tr} L = 0, \tag{13}$$

$$B = \tfrac{1}{2}\dot{c} \times y + \tfrac{1}{3}(Ly) \times y, \tag{14}$$

$$U = \nabla \times (\chi B) = \chi(\dot{c} + Ly) + \nabla\chi \times B, \tag{15}$$

$$p = 0, \qquad f = \partial_t U + (U \cdot \nabla)U - \nu\Delta U. \tag{16}$$

The prescribed motion determines the fixed external force and its spatial derivative; both are independent of learned parameters. The cutoff curl term is retained at box faces, edges and corners. Two properties of this instance matter for the diagnostics below. The motion is confined to $t \in (\frac{1}{4}, \frac{3}{4})$; at all other times $\dot{c}$, L and their derivatives vanish, so U, f and their derivatives vanish identically. During the motion the plateau $\chi = 1$ carries the affine velocity $U = \dot{c} + Ly$ with $\Delta U = 0$, and because L is diagonal its force $f = \ddot{c} + (\dot{L} + L^2)y$ is a gradient; the rotational part of the forcing lies in the cutoff layers. A CPU check of the frozen source confirms both properties to rounding [21]. Numerical choices belong to this project's single-box instance, which is a constructed forced-flow problem under stated conditions rather than a general formula for arbitrary Navier–Stokes data or a fluid regularity theorem.

The model uses viscosity $\nu = 0.01$, cutoff width $\eta = 1/32$, and the affine/tanh network widths 4–128–128–128–128–4 (50,692 parameters). Its outputs represent a three-component curl potential and pressure, with scales 0.01 and 0.1 respectively. The prescribed moving-box geometry supplies input features and a compact gate; the model and particle right-hand side use the learned potential, without inserting the exact interior velocity. The curl construction enforces divergence freedom in real arithmetic; numerical divergence is still measured.

For a scalar potential intermediate, the retained raw derivative state is

$$S_{36}(z) = (z, g, H, T, Q, m, n, K), \qquad Q_{ij} = \partial_{ij}\Delta z, \tag{17}$$

where g, H and T are the spatial gradient, symmetric Hessian and symmetric third derivative; $m = \partial_t z$, $n = \nabla\partial_t z$ and $K = \nabla^2\partial_t z$. These blocks have $1+3+6+10+6+1+3+6 = 36$ independent

channels. They supply the curl velocity, transient momentum residual and its gradient, including contracted fourth spatial derivatives of the potential. For comparison, a complete fourth-order jet in the four variables (x_1, x_2, x_3, t) has $\binom{8}{4} = 70$ coefficients. C36 is a partial-jet contract for this consumer, not a minimality result.

The canonical objective contains 4,096 interior, 512 initial, 384 boundary and seven pressure-gauge points. All 4,999 points and all 50,692 parameters are included. Frozen physical normalization and the original global family denominators are retained, including during chunk accumulation. FP64 and disabled TF32 are fixed. This campaign's two stored initializations are 361901 and 361902; they are not two-dimensional formal seeds. Source and calibration hashes and the complete protocol are part of the artifact [21].

6.2 Execution variants and numerical qualification

Three successive execution variants share the C36 algebra. (i) A packed-gather revision transposes symmetric unpacking through four cached integer gathers, keeping dense 9- and 27-entry activation storage and local state-autograd nonlinear pullbacks. (ii) A packed activation computes the six Hessian and ten third-derivative entries directly, and a manual analytic transpose reverses the activation's packed contractions and the actual stable coefficient program. (iii) The same variant with the packed forward compiled in both timing arms and the candidate's activation transpose compiled. Gate and curl nonlinear pullbacks still use local state autograd in all variants; this is not a fully manual adjoint for the entire model. For $y = \tanh z$ and $e = \exp(-2|z|)$, the frozen coefficient convention is

$$a_1 = 4e/(1+e)^2, \qquad a_2 = -2ya_1, \tag{18}$$

$$a_3 = 4y^2a_1 - 2a_1^2, \qquad a_4 = 16ya_1^2 - 8y^3a_1. \tag{19}$$

The transpose differentiates these evaluated operations, including the rounded tanh derivative and the exponential tail, rather than substituting an idealized fifth derivative at saturated values. Regression checks include values 0, $\pm 10^{-16}$, $\pm 10^{-12}$, ± 20 and ± 40, with cotangents scaled to 10^8.

The reference is a fresh native coordinate-AD evaluation on H100 at all 4,999 canonical points, for both initializations and all 50,692 parameters (52 comparisons, 140 saved arrays, maximum error 2.07227×10^{-5} of the unchanged $10^{-11} + 10^{-9}|g_{\mathrm{ref}}|$ budget). Later variants are checked against that preserved reference at original and changed parameters, in direct calls and graph replay, and against parameter AD. Variant (i) passes 48 comparisons. Variants (ii) and (iii) each pass 78 comparisons over 156 arrays with maximum error 2.35186×10^{-6} of the budget; both remote and local NumPy-only readers pass every comparison. A full canonical CPU qualification of the compiled analytic variant passes 42 comparisons (maximum 9.17612×10^{-6} of the budget). Exact initial tensor bytes come from the original checkpoints: regeneration under a newer Torch version was rejected after 1,804 of 50,692 entries differed by one ULP. No tolerance or scientific data was changed.

6.3 Paired complete-objective throughput

Each reported ratio is the geometric mean of matched baseline/candidate mean evaluation times. Two frozen initializations each have five alternating AB/BA pairs, for ten pairs per comparison. Candidate and parameter-AD windows last at least 60 seconds and contain at least three complete evaluations. Setup, compilation, warm-up and graph capture are excluded from these ratios. CUDA timings synchronize each complete evaluation. The 95% intervals use 10,000 paired bootstrap

resamples with seed 795912; they describe these timing pairs, not uncertainty across PDEs, datasets, hardware or training seeds.

Table 14: Complete C36 objective and all-parameter-gradient timing against the internal C36 parameter-AD control. Times are means over paired windows in seconds; the ratio is a paired geometric mean, not the ratio of these time means. Ten pairs per row. Rows 1–2 use variant (i), row 3 variant (ii) and rows 4–7 variant (iii); in every row both arms share the same forward and execution mode. Rows 6–7 come from one later registered batch on one H100: a repeat of row 4 and the same comparison with exact zero-gate constant folding in both arms. Different variants and devices are not pooled.

Device / mode	Candidate s	AD s	Ratio	95% interval
H100 eager, (i)	1.739582	1.692531	0.972935	0.970474–0.975924
H100 graph, (i)	0.484235	0.469551	0.969678	0.969317–0.970075
H100 packed graph, (ii)	0.328305	0.342941	1.044582	1.044130–1.045051
H100 compiled graph, (iii)	0.164484	0.217771	1.323962	1.323087–1.324762
CPU compiled, (iii)	1.865900	2.352072	1.265086	1.203149–1.327900
H100 compiled graph, (iii), repeat	0.166597	0.220791	1.325303	1.324919–1.325678
H100 compiled graph, (iii), folded	0.099377	0.139132	1.400047	1.399911–1.400178

Variant (i) is 2.782% slower in eager mode and 3.127% slower when both arms replay CUDA Graphs: integer gathers alone do not beat a strong parameter-AD control that shares the forward algebra. Computing the activation in packed symmetric storage with its analytic transpose turns this into a 4.458% gain, and compiling the packed forward in both arms plus the candidate transpose gives 1.324× (164.5 versus 217.8 ms per complete evaluation). The one-thread CPU result for variant (iii) is 1.265×. The parameter-AD control is a strong execution baseline but not an independent coordinate oracle, and no public library supplies this three-dimensional consumer; external three-dimensional baselines remain open (Section 8). An older comparison against a recursive native coordinate-AD implementation is recorded only in Appendix C, because its baseline is not competitive.

A later registered batch, whose protocol was committed before it ran, measures exact zero-gate constant folding on one H100. Folding evaluates the network only on rows where at least one of the 36 gate channels is nonzero, by exact equality: 2,933 of the 4,999 rows. Every other row keeps its parameter-independent forcing or target loss at the original weight; that loss is zero for this instance. Both arms fold the same rows.

The batch first runs the folded comparison and then an unfolded repeat on the same GPU. Both pass all 78 numerical comparisons, with a largest error of 3.0×10^{-6} of the budget, and the independent readers. The unfolded repeat gives 1.325 against 1.324 in row 4. With folding the ratio is 1.400 (1.3999–1.4002). The registered rule classifies this as an analytic advantage under folding, since the interval's lower bound exceeds 1.03.

Folding shortens an evaluation 1.68 times for the analytic candidate and 1.59 times for parameter AD; cost proportional to rows would give 1.70 (1.74 counting 128-row chunks). These factors compare sequential processes on the same GPU and are descriptive. That parameter AD gains less suggests it carries more cost that does not scale with rows. A folded analytic evaluation takes 99.4 milliseconds, against 220.8 for unfolded parameter AD. Folding depends on this instance's gate geometry and changes no training or acceptance result.

6.4 Frozen-budget training outcomes

Both H100 engines start from their original fresh initializations and execute 4,000 Adam updates plus 20 strong-Wolfe L-BFGS calls per seed. Adam has initial learning rate 10^{-3}, halved every 1,000 updates, betas (0.9,0.999) and epsilon 10^{-8}. L-BFGS uses learning rate one, history 20, at most 20 iterations and nominal 25 evaluations per call, zero gradient/change stopping tolerances and a 520-closure cap per seed. The complete-gradient checks run at frozen Adam checkpoints and after every L-BFGS call. Each engine passes all 48 scheduled saved-gradient comparisons. The original joint convergence gate requires loss at most 10^{-4} and gradient infinity norm at most 10^{-3}. The evaluated training sources predate variants (ii) and (iii).

Table 15: All four C36 training attempts finish the original finite budget. Actual L-BFGS closure counts (Adam evaluations excluded), final loss and full-gradient infinity norm are shown. Every attempt fails the joint convergence gate; no additional optimizer budget is silently appended.

Engine	Seed	Closures	Loss	$\|g\|_\infty$	Gate
Explicit	361901	444	0.00054021	0.00075071	Fail
Explicit	361902	438	0.00051678	0.00128521	Fail
Parameter AD	361901	449	0.00053870	0.00059044	Fail
Parameter AD	361902	443	0.00051532	0.00206306	Fail

6.5 Independent physical observations

The unchanged holdout seed 794701 supplies 4,096 stratified and 4,096 uniform-volume points, seven time slices with 256 points each, 512 initial points, 384 boundary points and seven pressure-gauge points. Native coordinate AD observes the final learned model on H100; NumPy reconstructs the physical metrics. Learned-velocity trajectories use SciPy DOP853 for 27 particles at 129 uniform output times plus the critical times, with two refinements (rtol, atol, max step) of $(10^{-9}, 10^{-11}, 1/64)$ and $(10^{-11}, 10^{-13}, 1/128)$. The exact velocity is used for error comparison, never as the trajectory right-hand side. Both stratified and uniform field gates and the trajectory gate must pass; time-slice and regional reports remain diagnostics.

Table 16: C36 final field failures. S/U denote stratified/uniform holdouts. Velocity errors are relative L2; momentum and corrected pressure use the original physical normalization. Row limits are velocity 0.01, momentum 0.01 and pressure 0.001. All displayed values exceed their corresponding gate; complete region and time-slice values remain in the result JSON.

Engine	Seed	Vel. S	Vel. U	Mom. S	Press. S	Press. U
Explicit	361901	0.020528	0.043743	0.014220	0.025935	0.003022
Explicit	361902	0.018808	0.034605	0.013856	0.022480	0.003407
Parameter AD	361901	0.020331	0.043126	0.014276	0.024746	0.002837
Parameter AD	361902	0.018497	0.034154	0.013832	0.019825	0.003094

For all four checkpoints, initial/boundary velocity and gauge errors are exactly zero. Stratified normalized divergence remains at most 2.01×10^{-14}, and divergence-gradient RMS at most 3.05×10^{-12}, against limits of 10^{-3} and 0.02; this follows from the curl construction rather than from training. Momentum-gradient RMS also passes its 0.02 limit. These satisfied gates coexist with failed velocity, stratified momentum, corrected-pressure and trajectory-endpoint criteria. Accurate

Table 17: C36 trajectories from the stricter original DOP853 refinement. Endpoint error must be at most 0.001; refinement change must be at most 10^{-7}. All refinement tests pass but all endpoint tests fail, so no row passes physical acceptance.

Engine	Seed	Endpoint	Refinement change	Gate
Explicit	361901	0.023166	7.2297×10^{-11}	Fail
Explicit	361902	0.029803	7.2587×10^{-11}	Fail
Parameter AD	361901	0.022762	7.2325×10^{-11}	Fail
Parameter AD	361902	0.029980	7.1729×10^{-11}	Fail

divergence and tightly converged ODE integration do not establish an accurate learned velocity field.

6.6 Diagnosis of the failed acceptance

Four observations indicate that the failure is limited by optimization rather than by derivative execution. First, the two engines reach nearly identical final losses from the same initialization (5.402/5.387 and 5.168/5.153 $\times 10^{-4}$), and all scheduled full-gradient checks pass. Second, the losses lie about five times above the 10^{-4} gate, not orders of magnitude away. Third, the budget is small relative to the problem: each C36 solve spends 438–449 L-BFGS evaluations on 50,692 parameters, whereas the accepted two-dimensional solves spend 2,208–5,259 L-BFGS evaluations after the same 4,000-update Adam phase on 4,547 parameters. Fourth, the stratified corrected-pressure error is 6.4–8.7 times the uniform-volume value, but this ratio mostly reflects how the two holdouts weight space: 4,033 of the 4,096 uniform-volume points lie outside the box support, where the reference flow vanishes, whereas the stratified holdout places 90% of its mass inside the support. The published observations of this protocol contain no regional breakdown; for the development run in Section 6.7 the largest pressure error lies in the box plateau rather than in the cutoff layers. These observations motivate a separately preregistered protocol with a larger L-BFGS budget; they do not license changing the completed protocol's budget, gates or seeds after observing its outcome. The Ethier–Steinman solves in Section 7, whose development run failed only its pressure gate at 100 L-BFGS blocks and passed once the block budget alone was raised, are consistent with this reading, although they use a different contract and problem.

6.7 Development run with a larger L-BFGS budget

A separately frozen development protocol changes only the L-BFGS budget, from 20 to 800 calls (closure cap 520 to 20,800). Data, model, normalization, Adam schedule, L-BFGS settings, seeds, training limits and every physical gate are those of the original protocol. Because seeds 361901 and 361902 were already observed, this is a reachability study rather than a formal time-to-accuracy comparison. Both engines train both seeds on one four-GPU H100 node: variant (iii) with its analytic activation transpose, and parameter AD over the same compiled packed forward, each with CUDA Graph replay. All complete loss/gradient checks pass at 28 check points per run, against dense parameter AD and, at the original parameters, against the preserved native reference, with maximum error 1.16×10^{-5} of the original budget. A decision rule was registered before the run, with the conditions (a) "Loss stays above 1e-4", (b) "Loss passes 1e-4 but physical errors stop falling" and (c) "Physical gates pass"; branch (c) would lead to a formal protocol with new seeds and both engines.

All four runs pass the training gate between L-BFGS calls 85 and 92, about 1,860–2,040 L-BFGS

Table 18: C36 development training with 800 L-BFGS calls. Gate is the first L-BFGS call at which a closure meets both training limits (loss $\leq 10^{-4}$, $\|g\|_\infty \leq 10^{-3}$); seconds count training time to that closure. Final values are after the full budget; s/eval is mean training time per complete evaluation, including optimizer work.

Engine	Seed	Gate call	Seconds	Final loss	Final $\|g\|_\infty$	s/eval
Analytic	361901	92	1,014	1.84×10^{-6}	2.9×10^{-5}	0.168
Analytic	361902	86	975	1.35×10^{-6}	1.9×10^{-5}	0.167
Parameter AD	361901	90	1,319	1.63×10^{-6}	4.4×10^{-5}	0.211
Parameter AD	361902	85	1,276	2.34×10^{-6}	1.7×10^{-6}	0.209

evaluations after Adam, far beyond the original 20 calls, and finish the budget with losses of 1.3–2.3$\times 10^{-6}$. On these development seeds the analytic engine reaches every loss threshold from 10^{-2} to 10^{-5} 1.30–1.33$\times$ sooner than parameter AD, with nearly identical evaluation counts. Both arms' training times include the same scheduled dense-AD checks and checkpoint writes, which bias these ratios toward one.

Table 19: Physical observation of the final C36 development checkpoints. S/U are the stratified and uniform holdouts; velocity is relative L2 and pressure the normalized corrected-pressure RMS. Limits: velocity 0.01, pressure 0.001, endpoint 0.001. The batch's own time guard stopped the parameter-AD observations after both holdouts, so that the reader and archive could finish within the allocation; they are partial.

Engine	Seed	Vel. S	Vel. U	Press. S	Press. U	Endpoint	Outcome
Analytic	361901	0.0013	0.0022	0.0025	0.0004	0.00107	Fail
Analytic	361902	0.0012	0.0023	0.0025	0.0003	0.00097	Fail
Parameter AD	361901	0.0011	0.0020	0.0021	0.0004	–	Partial
Parameter AD	361902	0.0015	0.0025	0.0033	0.0005	–	Partial

One complete physical observation takes about 99 minutes, so only the final checkpoints were observed in this batch (Table 19). These metrics are the observer's own NumPy consumer output; the later observation batches of Section 6.8 reproduce both analytic rows exactly, complete the two partial rows and re-read every observation with the independent reader. Compared with the original budget, stratified velocity error falls from 0.018–0.021 to 0.0011–0.0015, uniform velocity error from 0.034–0.044 to 0.0020–0.0025, stratified momentum RMS from 0.014 to 0.0009–0.0011, and the analytic trajectory endpoint error from 0.023–0.030 to 0.00097 and 0.00107. The uniform field gate passes in every run, although only 63 of its 4,096 points lie inside the box support; the initial and boundary values and the trajectory refinement checks pass in both complete observations. The stratified corrected pressure, 0.0021–0.0033 against 0.001, fails in every run, and the endpoint error of seed 361901 exceeds its limit by 7%. For the analytic runs the pressure error is largest in the plateau region inside the box (0.0042–0.0046), where the velocity error is also largest, rather than in the cutoff layers. The reader's mechanical classification is (b), because the training gate passes and no observation passes physical acceptance. The trend condition of (b), that physical errors stop falling, could not be evaluated from this batch, which observed only the final checkpoints; relative to the original budget every reported physical error fell, by 7–31 times per seed. Section 6.8 observes every saved checkpoint and finds the errors still falling at the end of the budget, so (b) does not apply either, and none of the three registered branches applies cleanly. The run shows that budget alone reaches the training gate but not, within 800 calls, the stratified pressure gate. A

formal protocol needs a larger budget or a changed problem definition, stated before it runs.

6.8 Physical errors along the training trajectory

Four further run-once batches observe every saved checkpoint of the development runs, from the untrained parameters through Adam updates 1,000–4,000 and L-BFGS calls 20–800 (52 saved points, 46 distinct parameter vectors), and 26 checkpoints of the continuation runs of Section 6.9. Each of the 72 observations uses the unchanged observer on every data plane: the stratified and uniform holdouts, seven time slices, the initial and boundary sets, and both trajectory refinements. Physical acceptance gates the two holdouts, the initial and boundary values and the finer trajectory refinement; the time slices are diagnostic planes evaluated with the same field limits. The independent NumPy reader re-reads every observation (117 saved-array checks each, 8,424 in total) and passes. Re-observing the two analytic development finals reproduces every metric of their earlier observation exactly.

Each batch chooses its observer chunk by a rule written before its calibration: the fastest chunk whose states agree with chunk 128 to 10^{-10} of their maximum magnitude within a peak-memory limit. On the 512 calibration points, chunk 512 is 3.9× faster than chunk 128, agrees to at most 1.8×10^{-11} absolute (7.9×10^{-12} relative) and peaks at 56.6 GB; chunks 1,024 and 2,048 exceed device memory. The first batch's 50 GB limit, set before this measurement, kept chunk 128 (about 100 minutes per observation), so its deadline guard stopped it after 8 of 46 planned observations. Later batches observed the remaining checkpoints with a corrected 60 GB limit and chunk 512 (median 31 minutes). Chunk equivalence rests on the calibration points; no full checkpoint was observed at both chunks.

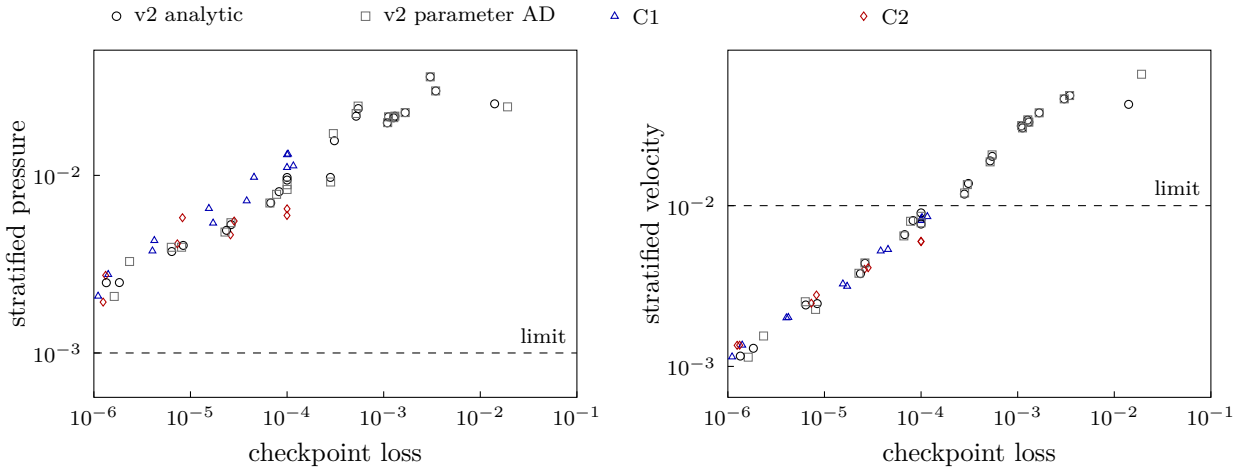


Figure 4: Stratified corrected pressure (left) and stratified velocity error (right) against the training loss at every original-problem checkpoint of the development runs (v2) and the continuation runs (C1, C2); 64 of the 72 observations. The untrained parameters, which have no logged loss, and the six checkpoints taken during the $w = \frac{3}{4}$ stage are omitted. Loss is the last logged training evaluation at the checkpoint; for Adam checkpoints it precedes the saved update by one step. Dashed lines are the acceptance limits.

No observation passes physical acceptance. The initial and boundary values vanish identically, because the gate vanishes there. The uniform holdout passes from L-BFGS call 200 on (loss at most 2.8×10^{-5}) and fails before; the trajectory endpoint passes only at three final checkpoints. The stratified holdout fails at every checkpoint; at the final checkpoints only its corrected pressure

fails, 1.9–3.3 times the limit, while velocity, momentum and divergence pass. The diagnostic time slices split the same way. The four slices at which the box is at rest ($t = 0, \frac{1}{4}, \frac{3}{4}, 1$), where the exact velocity and force vanish identically, pass at every checkpoint. The three inside the motion window fail at every checkpoint, by at least 2.2 times on pressure.

Along each development run the physical errors fall with the loss (Fig. 4). Rank correlations between log loss and log error are 0.95–0.96 for stratified pressure and 0.99–1.00 for stratified velocity; over the L-BFGS checkpoints the log–log slopes are 0.31–0.41 and 0.45–0.50. In all four runs pressure rises once, from the end of Adam to L-BFGS call 20, while velocity falls. The only other increase is in velocity, at an Adam-phase loss spike in the parameter-AD run of seed 361902. Between the last two checkpoints (L-BFGS calls 400 and 800, loss 2.7–5.0 times lower), stratified pressure still falls by factors 0.53–0.83 and velocity by 0.48–0.61. The trend condition of branch (b) therefore does not hold at the end of this budget: the physical errors have not stopped falling; they fall more slowly than the loss. If the observed pressure slopes persisted, the pressure gate would need a loss 6–47 times below the final values. Section 6.10 tests this with six times the L-BFGS budget: the loss and the errors stall instead. The two development engines, whose trajectories differ only by rounding, end at stratified pressures of 2.49 and 2.08×10^{-3} (seed 361901) and 2.49 and 3.28×10^{-3} (seed 361902), which sets the scale of trajectory sensitivity at equal budget.

6.9 Temporal-smoothness continuation

The development runs suggest that the motion window is the hard part of the problem. A continuation schedule therefore starts from a slower, smoother motion and returns to the original problem. The box motion is driven by one schedule $s(t) = \sigma(2t - \frac{1}{2})$, where σ is the smooth unit switch of the construction, so the box moves for $t \in (\frac{1}{4}, \frac{3}{4})$. A transition width w generalizes it to

$$s_w(t) = \sigma\left(\frac{t}{w} - \frac{1}{2w} + \frac{1}{2}\right), \qquad \text{moving for } t \in \left(\tfrac{1}{2} - \tfrac{w}{2}, \tfrac{1}{2} + \tfrac{w}{2}\right), \tag{20}$$

with the same displacement and dilation, centered at $t = \frac{1}{2}$, and with k-th time derivatives scaling as w^{-k}; $w = \frac{1}{2}$ is the original problem. Because the model gate contains the schedule speed, a width changes the geometry features, the gate and the forcing; value targets remain zero, which is checked. The original problem is reproduced bit for bit at $w = \frac{1}{2}$. At $w = \frac{3}{4}$ both engines agree with native coordinate AD on the changed objective (six CPU tests). During training each stage is entered with a fresh compiled engine and CUDA Graph. It is checked against dense parameter AD at the entry parameters, after a parameter change, and again after garbage collection and allocator churn. Physical acceptance always uses the original problem.

Two continuation schedules spend the development budget (4,000 Adam updates and 800 L-BFGS calls with the same settings) differently. C1 runs 1,000 Adam updates at $w = 1$, 1,000 at $w = \frac{3}{4}$, then 2,000 Adam updates and all 800 L-BFGS calls at $w = \frac{1}{2}$. C2 runs 2,000 Adam updates at $w = 1$, 2,000 Adam updates and 100 L-BFGS calls at $w = \frac{3}{4}$, then 700 L-BFGS calls at $w = \frac{1}{2}$. Each uses the same two initializations and is paired with the analytic development run of the same seed, the $w = \frac{1}{2}$ baseline. Loss thresholds count only evaluations of the original problem, while evaluations and seconds are counted from the start. The rule registered before the run reads: helps, "both schedules or one schedule reach every threshold sooner and the final physical errors (stratified pressure in particular) are lower than the baseline on both seeds"; no effect, "differences within the seed-to-seed spread of the baseline"; hurts, "thresholds reached later or final errors higher on both seeds".

The first attempt failed in all four lanes at its first in-training check, after 100 Adam updates. The graph-capture helper retains the graph and its outputs, but not the engine that owns the

prepared data, and the new trainer had kept that engine only in a stage-local variable. Graph replays after stage entry therefore read released memory, and the loss became negative from the second update on. The batch archived itself and is preserved. The repair keeps the engine with its replay and adds the post-churn check and a non-negative-loss guard. It also gives the CPU self-test a replay that holds only a weak reference to the engine, so the same error now fails the self-test (verified by a negative control). The second attempt changes only the trainer.

Table 20: Smoothness continuation against the $w = \frac{1}{2}$ baseline (development analytic run). Each entry counts complete evaluations, or training seconds, from the start to the first original-problem loss at or below the threshold. Seconds include the continuation runs' stage rebuilds (about 65 s each) and every scheduled check.

Seed	Schedule	10^{-3}	3×10^{-4}	10^{-4}	3×10^{-5}	10^{-5}
Evaluations						
361901	baseline	4,042	5,144	6,039	8,077	12,129
361901	C1	2,577	4,538	5,128	6,704	9,975
361901	C2	6,239	6,311	6,571	8,010	12,192
361902	baseline	4,030	5,048	5,891	7,600	10,955
361902	C1	2,764	4,576	5,200	7,012	9,903
361902	C2	6,237	6,306	6,585	8,262	11,622
Seconds						
361901	baseline	677	864	1,014	1,359	2,045
361901	C1	564	889	989	1,254	1,826
361901	C2	1,190	1,202	1,246	1,488	2,195
361902	baseline	666	834	975	1,262	1,821
361902	C1	593	896	1,003	1,309	1,800
361902	C2	1,186	1,197	1,244	1,529	2,118

All four continuation runs complete the budget and pass every scheduled complete-gradient check (32 per run, at most 1.2×10^{-5} of the original budget). They reach the training gate and end with losses of 1.11 and 1.41×10^{-6} (C1) and 1.25 and 1.32×10^{-6} (C2), against 1.84 and 1.35×10^{-6} for the baseline (Table 20). C1 reaches every registered threshold in 8–36% fewer evaluations than the baseline on both seeds, but not always in fewer seconds. It is later at 3×10^{-4} on both seeds and at 10^{-4} and 3×10^{-5} on seed 361902, each time by less than the 130 seconds of its two stage rebuilds, which precede every threshold. C2 is later at every threshold on both seeds in seconds, and at nine of ten seed–threshold pairs in evaluations, because its first original-problem evaluation comes after 6,200 evaluations. The coarser level 10^{-2} is not a registered threshold; C1 reaches it only after switching to the original problem at evaluation 2,001, against about 560 for the baseline.

At equal loss the schedules end at different physical errors (Table 21). At the first closure that meets the training gate, C2's stratified pressure, velocity and endpoint errors are 0.61–0.69, 0.67–0.78 and 0.40–0.45 times the baseline's on both seeds, outside the 6–14% pressure difference between the two development engines at the same point, while C1's pressure is 1.18–1.35 times higher. The difference shrinks as training continues. At the final checkpoints the stratified pressure of both schedules is lower than the baseline's on seed 361901 (2.09 and 1.93 against 2.49) and higher on seed 361902 (2.77 and 2.74 against 2.49), changes of 10–22% that are comparable to the 16–32% difference between the two engines. Every final checkpoint fails the stratified pressure gate. The failing acceptance tests differ only in the trajectory endpoint, which C2 passes on seed 361902 and C1 fails on both seeds.

The width-$\frac{3}{4}$ parameters transfer to the original problem much better than the fields themselves

Table 21: Physical errors of the continuation runs on the original problem, $\times 10^{-3}$. "Gate" is the first closure that meets the training gate (loss $\leq 10^{-4}$ and $\|g\|_\infty \leq 10^{-3}$), reached at L-BFGS calls 86–92 (baseline), 51–55 (C1) and 117–118 (C2); the comparison there is matched by loss, not by budget. S: stratified holdout; plateau: its region inside the box; endpoint: trajectory endpoint error. Limits: pressure 1, velocity 10, endpoint 1. Failed lists the failing acceptance tests at the final checkpoint: S (stratified pressure) and T (trajectory endpoint).

		Gate			Final				
Seed	Schedule	Press. S	Vel. S	Endpoint	Press. S	Plateau	Vel. S	Endpoint	Failed
361901	baseline	9.73	9.02	11.1	2.49	4.65	1.30	1.07	S, T
361901	C1	13.1	8.27	8.29	2.09	3.11	1.15	1.12	S, T
361901	C2	5.95	6.00	5.01	1.93	3.14	1.35	1.01	S, T
361902	baseline	9.41	7.69	10.2	2.49	4.17	1.16	0.97	S
361902	C1	11.1	8.09	10.8	2.77	4.35	1.36	1.12	S, T
361902	C2	6.48	5.97	4.07	2.74	4.39	1.35	0.89	S

would suggest. Observed on the original problem at the end of the $w = \frac{3}{4}$ stage, they have stratified velocity errors of 0.049–0.059. Yet the exact $w = \frac{3}{4}$ and $w = \frac{1}{2}$ velocity fields differ by 0.81 in the same norm, and by 0.99 on the slice $t = \frac{3}{8}$. The gate carries the schedule speed and the potential is learned in moving-box features, so most of the width dependence lies outside the learned function. C2's 100 L-BFGS calls at $w = \frac{3}{4}$ lower that stage's loss to 1.4–1.7×10^{-5} but leave the transferred velocity error at 0.049.

Against the registered rule, the continuation does not help: neither schedule reaches every threshold sooner while lowering the final errors on both seeds. Read per schedule, C2 meets the hurts clause through its thresholds, which come later at all five on both seeds in seconds and at nine of ten in evaluations. C1 meets neither clause. It is not helpful, because its final stratified pressure is higher on seed 361902 and not all its threshold times in seconds are shorter. Nor is it harmful, because all its evaluation counts are lower and its final errors are not higher on both seeds. The no-effect band is degenerate, because both baseline seeds end at the same stratified pressure to three digits, while two numerically equivalent engines differ by up to 32%. The rule also did not state whether thresholds count evaluations or seconds. As for the development run, no branch applies cleanly to C1. Within this budget, a smoother early problem moves the training trajectory, and C2 meets the training gate closer to the physical solution. Neither schedule changes which acceptance tests fail, however: the stratified pressure inside the moving box during the motion remains the binding gate.

6.10 A larger L-BFGS budget

Section 6.8 leaves one direct question: does more optimizer budget reach the stratified pressure gate? Two further studies were registered to answer it.

A cancelled resumed design. The first design (protocol v4) resumed the final checkpoints of the C2 and baseline runs and added 4,000 L-BFGS calls with a fresh L-BFGS history. Its trainer verified each start checkpoint's hashes and refused a tampered start. Every qualification check passed, and each source run's final loss was reproduced bit for bit or within 4.2×10^{-11} of the original budget.

After 13.5 minutes, before any physical observation, the batch was cancelled on optimizer progress alone. A fresh history started at a loss of about 10^{-6} falls into a cycle: each trial step

$t = 1$ raises the loss to 1.3–1.9$\times 10^{-6}$, and the strong-Wolfe search then accepts a tiny step. Over 100 calls the loss fell by 0.2–0.7%, against about 32% over the last 100 calls of the source runs. Fresh histories started earlier at losses of 10^{-2} and 10^{-3} had behaved normally. The resumed design would therefore have measured the restart, not the budget; its partial outputs are archived. The source runs also showed that the continuation runs had already stalled at 1.1–1.4$\times 10^{-6}$ by call 800 (0.02–0.46% per 10 calls), while the baseline runs were still falling by 3.4–3.7% per 10 calls. The smoothness arm was therefore dropped.

Protocol v5. The second design keeps the development recipe unchanged, trainer and driver included, and extends the L-BFGS budget from 800 to 4,800 calls in one continuous run per seed and engine, so the L-BFGS history never restarts. Both engines train both seeds on one four-GPU H100 node. A rule registered before the run, implemented as code and tested on synthetic outcomes, classifies the result from the physical observations at 1,600, 2,400, 3,200 and 4,000 calls and at the end:

- E1: all four runs pass;
- E2: exactly one engine passes on both seeds;
- E3: some runs pass;
- E4: no run passes and the errors still fall;
- E5: no run passes and the errors have stopped falling, meaning a final stratified pressure at least 0.95 times its value at 4,000 calls on both seeds of an engine.

E1 or E2 would lead to a formal fresh-seed time-to-accuracy protocol. The runs do not reproduce the development runs bit for bit: the first difference appears in the last digit of the loss between Adam updates 3 and 8, a reduction-order effect, so call 800 was observed anew.

Table 22: C36 with six times the L-BFGS budget (protocol v5). Loss and stratified corrected pressure at L-BFGS call 800 (the development budget) and at the end of 4,800 calls; plateau is the stratified region inside the box; velocity is the stratified relative L2 error; endpoint is the trajectory endpoint error. Physical values $\times 10^{-3}$; limits: pressure 1, velocity 10, endpoint 1. Failed lists the failing acceptance tests at the end: S (stratified pressure) and T (trajectory endpoint).

		Loss $\times 10^6$		Press. S					
Engine	Seed	800	4,800	800	4,800	Plateau	Vel. S	Endpoint	Failed
Analytic	361901	1.830	1.607	2.40	2.36	4.29	1.21	0.90	S
Analytic	361902	1.383	1.338	2.57	2.52	4.52	1.16	1.00	S
Parameter AD	361901	1.749	1.628	2.26	2.17	3.96	1.27	1.02	S, T
Parameter AD	361902	1.499	1.432	3.03	2.97	5.89	1.19	0.86	S

All four runs complete the 4,800 calls within the time guard and pass all 60 scheduled complete-gradient checks (at most 1.1×10^{-5} of the original budget). The loss stalls soon after call 800 (Table 22). Between calls 800 and 4,800 it falls by only 3–12%, and after call 1,200 by about 0.2–0.3% per 400 calls. No run reaches a loss of 10^{-6}; the threshold 3×10^{-6} is reached after 16,077–18,924 evaluations.

A separate run-once batch observes every run at calls 800, 1,600, 2,400, 3,200, 4,000 and 4,800 on all planes (24 observations, chunk 512). The independent reader passes all 2,808 checks. The physical errors stall with the loss:

- Stratified corrected pressure changes by 1.5–3.8% between calls 800 and 4,800 and ends at 2.2–3.0 times its limit.
- The plateau error inside the box stays at 4.0–5.9 times the limit.
- The uniform holdout passes at every checkpoint.
- The trajectory endpoint stays at 0.86–1.02 times its limit and passes at 12 of the 24 checkpoints.

No checkpoint passes physical acceptance. The registered outcome is E5: no run passes, and both engines have stopped falling, with final stratified pressures 0.998 times their values at 4,000 calls. The rule therefore makes no formal claim.

This answers the trend question of Section 6.8. The errors were still falling at the development budget of 800 calls. With this optimizer, however, they stop shortly afterwards, together with the loss, at a point whose gradient infinity norm is $1.2\text{–}4.6 \times 10^{-6}$. The extrapolation in Section 6.8 does not hold, and the remaining stratified pressure error is not a budget limit of this optimizer at this scale. In the terms of the development rule this is branch (b): reaching the stratified pressure gate needs an explicitly changed problem definition, such as different sampling, weights or capacity, in a new protocol. As a development observation only, parameter AD needed 1.28–1.36 times the seconds of the analytic engine to each loss threshold from 10^{-3} to 3×10^{-6}, as at the shorter budget.

Where the remaining pressure error lies. A post-hoc decomposition of the saved native states of the four final checkpoints (CPU only) splits the stratified corrected-pressure error by region; it reproduces every recorded regional and global value exactly. Within the motion window the plateau error is 4.4–6.6 times its limit. A per-time offset explains 27–70% of its variance, and an offset plus a term linear in y explains 64–76%. The cutoff layers carry 1.2–3.4 times the limit, at most about 30% of it a per-time offset. With the plateau error set to zero, the layers alone would leave a stratified value of 1.5–1.9 times the limit, so changing only the weight or sampling of the plateau cannot reach the gate. The layer forcing sets this scale. With $\eta = 1/32$ the layer force and force-gradient component RMS are 196–234 and $7.0\text{–}7.2\times10^4$, against 2.0 and 10.6 in the plateau; with $\eta = 1/16$ they are four to five and nine to eleven times smaller. A wider cutoff defines a different instance rather than a repair of this one; Section 6.11 tests it.

6.11 A wider cutoff (protocol v6)

The post-hoc above motivated one more registered development study, on a different instance: the same single-box construction with the cutoff width doubled to $\eta = 1/16$. Every other element follows the development recipe: the box motion, the sampler seeds and point counts, the model and its features, the rule that derives the scales from the calibration, the physical limits, the Adam schedule, the L-BFGS settings with 800 calls, the stored initial parameters, both engines and the observer. The registered decision applies within this instance only. F1: some run passes physical acceptance. F2 (near miss): some complete final observation exceeds no limit by more than 1.5 times. F0: a run lacks a complete final observation, with no scientific reading. F3: otherwise.

The calibration and training generators are the originals with the cutoff width as an argument; at $\eta = 1/32$ they reproduce the frozen inputs to 2.8×10^{-14} of each array's maximum. At $\eta = 1/16$ ten calibration points pass the 80- and 160-digit source checks, and the prepared geometry agrees with native coordinate AD. The scales follow the instance. The stratified velocity RMS falls from 0.377 to 0.303, so the absolute corrected-pressure tolerance tightens from 1.42×10^{-4} to 9.2×10^{-5}, while the force and force-gradient scales fall from 164 to 38.1 and from 50,451 to 5,252. The preserved

full native reference belongs to the $\eta = 1/32$ data, so it supplies only the stored initial parameters. Instead, the eager engine is checked against fresh native coordinate AD on a 131-point interior subset that covers every region, plus every value point, and the compiled and graph engines against dense parameter AD. All start-up checks pass within 10^{-6} of the original budget and all 116 saved checks within 1.7×10^{-5}.

Table 23: C36 on the wider-cutoff instance ($\eta = 1/16$, protocol v6; 800 L-BFGS calls). Gate is the L-BFGS call of the first closure meeting the training gate. Pressure columns are the final stratified corrected pressure, its plateau region and, post hoc, the value the cutoff layers alone would leave with the plateau error set to zero, all over the limit; velocity is the stratified relative L2 error $\times 10^{-3}$ (limit 10) and endpoint the trajectory endpoint error $\times 10^{-3}$ (limit 1). Failed lists the failing acceptance tests: S (stratified pressure) and T (trajectory endpoint).

				Pressure / limit					
Engine	Seed	Gate	Loss $\times 10^6$	S	Plateau	Layers	Vel. S	Endpoint	Failed
Analytic	361901	149	4.227	2.89	4.84	2.20	1.81	0.80	S
Analytic	361902	110	2.767	2.16	3.37	1.73	1.46	0.80	S
Parameter AD	361901	152	3.490	2.58	4.10	2.04	1.59	0.62	S
Parameter AD	361902	111	3.002	2.04	3.02	1.67	1.56	1.09	S, T

The batch ran once on one four-GPU H100 node, and every run trained, was observed at its final and first training-gate checkpoints, and was read independently. All four runs reach the training gate after 110–152 L-BFGS calls and end at losses of 2.8–4.2$\times 10^{-6}$ (Table 23). At the final checkpoints the stratified corrected pressure, 2.0–2.9 times its limit, is the only failing test in three runs. The fourth also misses the trajectory endpoint by 9%. Velocity, momentum, divergence, the uniform holdout and the initial and boundary values pass. The first training-gate checkpoints fail more widely (stratified pressure 9.1–11.5 times its limit, trajectory endpoint 4.8–9.0 times). The registered outcome is F3: no run passes, the best final observation is 2.04 times its worst limit, and all four final observations are complete.

Doubling the cutoff lowered the layer forcing four- to fivefold, but not the normalized pressure error. That error stays where it was at $\eta = 1/32$ after 800 calls (2.3–3.0 times its limit), and the cutoff layers alone still leave 1.7–2.2 times the limit. Its absolute size is somewhat smaller, but so is the tolerance. The remaining stratified pressure error is therefore not set by the magnitude of the layer forcing alone. As a development observation only, parameter AD needed 1.25–1.34 times the seconds of the analytic engine to each loss threshold from 10^{-2} to 10^{-5}, and 1.26–1.27 times per evaluation.

What drives the pressure error (post hoc). Because the exact pressure vanishes, every learned pressure gradient is error. Pointwise, $\nabla p = R - \delta I$, where R is the observed momentum residual and δI the inertia error of the learned velocity, which vanishes for the exact field. A CPU decomposition of the saved native states of the v5 and v6 final checkpoints reproduces the observer's regional values. It checks this identity and its divergence to within 4×10^{-12} of the largest value, and gives three findings. First, the pressure gradient is not forced by the velocity error: its cosine with $-\delta I$ lies between -0.05 and 0.07 in every region, at the sampling noise floor of 0.02–0.03. Second, δI dominates the residual at 3–15 times the pressure gradient, so the pressure carries 4–11% of the residual energy in the plateau and 0.5–1.3% in the cutoff layers. The plateau itself holds 0.6–2.5% of the stratified momentum energy. Third, in the Poisson balance $\Delta p = \nabla \cdot R - \nabla \cdot \delta I$ the velocity term is 0.3–0.7 times Δp in the plateau. In the layers both divergences are large and

nearly cancel, so the balance is not interpretable there. The remaining pressure error is therefore an independent component that the loss weights at about one percent. A more accurate velocity would not by itself remove it. Reaching the gate needs a term that weights pressure on its own scale, or an optimizer that resolves such a small component.

6.12 A dense quasi-Newton optimizer (protocol v7)

Protocol v7 tests the second of these options on the original instance ($\eta = 1/32$). It keeps the development recipe, including the stored initial parameters, the data, the 4,000 Adam updates, the objective and every numerical and physical limit, and replaces only the L-BFGS phase by a self-scaled Broyden update (SSBroyden) with a dense FP64 inverse Hessian on the GPU (20.6 GB) [18, 19]. Each iteration costs one dense matrix–vector product, and a strong-Wolfe line search from a unit step keeps the L-BFGS settings. The update is skipped without curvature, and the inverse Hessian is reset after a non-descent direction or a failed line search. Unit tests check it against the textbook inverse BFGS update and an independently written implementation of the second paper's definitions. The training guard is 4,000 seconds. The decision reuses the v6 outcomes in order (F1 pass, F2 near miss, F0 incomplete, F3 fail).

The registered batch had four H100 lanes: two fresh runs from the stored initializations and two continuations of the v5 analytic final checkpoints. Its outcome could not be retrieved, and the continuation start checkpoints became unavailable. Those two arms were never run elsewhere, and v7 as registered remains unresolved. A separate registration then ran the two fresh runs on NVIDIA GH200 (ARM host, one GPU per seed, Torch 2.14.1). The optimizer, the trainer, the guard, the reader and the rules were unchanged, and the decision applies to these two runs. A qualification batch first compared eager, compiled and graph objectives at both stored initializations with the preserved full native reference, and a changed parameter vector with dense parameter AD. All 32 comparisons were within 2.3×10^{-6} of the original budget. It also allocated the full inverse Hessian.

Table 24: C36 with dense SSBroyden after the unchanged 4,000 Adam updates (protocol v7, fresh arm, GH200; 4,000-second training guard). Iterations and evaluations count the SSBroyden phase only. Physical values are the final observation over each limit; S is stratified, U uniform. Endpoint is the trajectory endpoint error over its limit. Every other acceptance test is further below its limit.

					Pressure / velocity / limit			
Seed	Iter.	Eval.	Loss	$\|g\|_\infty$	Press. S	Press. U	Vel. S	Endpoint
361901	14,412	23,051	2.9×10^{-13}	5.4×10^{-8}	9.9×10^{-4}	1.1×10^{-4}	5.9×10^{-5}	3.2×10^{-4}
361902	14,637	23,187	1.7×10^{-13}	3.7×10^{-8}	9.5×10^{-4}	1.3×10^{-4}	4.6×10^{-5}	2.1×10^{-4}

Both runs trained for the full guard without a reset or a skipped update, and every saved complete-gradient check is within 5.3×10^{-5} of the original budget (78 comparisons per run). The carried inverse-Hessian product agrees with an explicit product to 1.5×10^{-10} at all 14 checks. The loss falls to 1.7–2.9×10^{-13}, against 1.3–1.6×10^{-6} after 4,800 L-BFGS calls in v5. Both final observations pass every acceptance test (Table 24). Stratified pressure, the test that every earlier C36 run failed, is now about 10^{-3} of its limit, and it is still the tightest test. The registered outcome is F1 for both runs; the in-batch reader and a separate CPU readback of the archive both recompute it.

The registered secondary readout counts evaluations to each loss threshold, including the Adam phase, against the v5 analytic run of the same seed. The ratio v7/v5 is 1.00 at 10^{-3}, 0.71–0.73 at 10^{-4}, 0.38–0.43 at 10^{-5} and 0.26–0.30 at 3×10^{-6}. v5 never reaches 10^{-6}; v7 reaches 10^{-7} after

5,684–5,718 evaluations. These are evaluation counts on different hardware, not a time ratio. Each SSBroyden iteration takes 0.23–0.24 seconds on GH200 with 1.58–1.60 line-search evaluations. The training gate (loss 10^{-4}, gradient 10^{-3}) is met after about 600 training seconds. The gate-first observation was skipped by the observation budget, because each final observation took 2,994–3,307 seconds, so this batch does not show when physical acceptance was first reached. Both seeds were seen before. These are development runs, and this batch makes no formal claim.

6.13 Prospective time to verified accuracy

A formal cohort followed, registered before any of its runs and committed publicly. It uses five previously unused seeds from a fixed SHA-256 rule and a new common physical holdout seed. The v7 fresh recipe is unchanged, and each seed runs as an independent single-GPU job on GH200. For each new initialization the run checks the eager objective against fresh native coordinate AD on the fixed 131-point subset. It also checks the eager, compiled and graph objectives against fresh dense parameter AD. The first physical inspection comes at the first optimizer boundary after 1,000 active training seconds. A pass stops the run; otherwise training continues from the same optimizer state to the guard and is inspected again. The time to verified accuracy (TTA) runs from trainer entry, through every import, check, compilation and observation, to the first passing physical report. A pass becomes valid only after an independent CPU audit of the sealed archive. The cohort qualifies only if all five audits pass, and the record reports a median TTA only in that case. There is no paired baseline and no speedup claim.

Table 25: Formal C36 TTA cohort (five new seeds, one GH200 each). Gate is the active training second at which the training gate is first met. At inspection: active training seconds and SSBroyden iterations at the first physical inspection, and its loss. Observation is the duration of that inspection. Candidate TTA is the recorded time to a passing physical report; it is not qualified, because no audit passed. S press. is the stratified corrected pressure over its limit, the tightest test.

Seed	Gate (s)	Insp. (s)	Iter.	Loss	Obs. (s)	Cand. TTA (s)	S press.	Audit
1212382065	594	1,003	1,985	9.7×10^{-9}	3,195	5,491	0.267	failed
392322075	597	1,004	1,942	8.5×10^{-9}	3,134	5,581	0.229	failed
422010988	600	1,004	1,975	–	stopped	–	–	not run
1127789372	594	1,003	1,903	6.7×10^{-9}	3,073	5,228	0.215	failed
949592700	590	1,003	2,069	1.1×10^{-8}	3,173	5,319	0.206	failed

All five runs passed their start-up checks, within 1.2×10^{-5} of the original budget for every saved check. They met the training gate after 590–600 active seconds, and were first inspected after about 1,003 seconds and 1,903–2,069 iterations (Table 25). Four inspections completed, and all four pass physical acceptance at the first inspection: stratified pressure is 0.21–0.27 of its limit, stratified velocity 7.5–8.5 $\times 10^{-5}$ in relative L2, and the trajectory endpoint 0.03–0.05 of its limit. Their candidate TTAs are 5,228–5,581 seconds. About 1,006 of those seconds are training and 3,092–3,217 the observation, including moving the inverse Hessian to host memory and back; 1,128–1,417 seconds go to imports, compilation, and the native-subset and dense-AD checks of the new initialization. The fifth run was cancelled by the operator during its first observation and stays in the denominator. The five jobs used 7.66 allocated GPU-hours.

All four frozen audits fail at the same assertion. The auditor recomputes the physical consumer's raw arrays from the saved native states on an x86 CPU and requires agreement within a relative tolerance of 10^{-10} against the values saved on the GPU host. The spatial residual gradient fails this test in 2–5 of 36,864 stratified elements, and in up to four time-slice elements. The stratified

global metrics, recomputed just before the failing assertion, agree with the report. The audit stops there, so the remaining checks of these four runs never ran. The registered outcome is therefore *not qualified*: 0 of 5 qualified runs (Wilson 95% interval 0–0.43), no median and no range.

Why the audits fail (post hoc). The spatial residual gradient is a sum of terms that reach 1.4×10^5 and cancel to about 10^{-2}. A tolerance relative to the result is therefore far below the rounding error of its terms. On every dataset of the four observations, the ARM–x86 difference is at most four units in the last place of the largest term. It is at most 0.13 of the standard bound $2\gamma_{10} \sum |\text{terms}|$ for two independently rounded evaluations, and no element exceeds that bound. On the four resting time slices this array agrees bit for bit. The failure thus comes from the auditor's tolerance, not from the observer or the physics. The auditor's CPU regression tests ran producer and consumer on the same x86 host and could not expose it. This diagnosis does not change the registered outcome. A qualified cohort needs a new registration with fresh seeds and a tolerance scaled by the size of the summed terms.

6.14 A second, qualified cohort

That registration followed, before any of its runs. It changes three things: five new seeds from the same rule under a new label, excluding every seed seen before; a new holdout seed; and a second revision of the auditor. The revised auditor requires the four summed raw arrays to agree within the a-priori bound $2\gamma_{16} \sum |\text{terms}|$, and the scalar metrics built from them within the bound that the triangle inequality implies. Every other check, and every other element of the protocol, is the first cohort's. Before registration, the revised auditor ran end to end on the four complete archives of the first cohort and passed. Every check after the first auditor's failure point therefore ran on ARM-produced data before any new run. The largest differences were 0.08 of the array bound and 0.0014 of the metric bound. That validation does not change the first cohort's outcome, and the two cohorts are not pooled.

Table 26: Second formal C36 TTA cohort (five new seeds, one GH200 each; columns as in Table 25). All five runs pass physical acceptance at the first inspection, and all five independent audits pass, so each TTA is qualified.

Seed	Gate (s)	Insp. (s)	Iter.	Loss	Obs. (s)	TTA (s)	S press.	Audit
2013091829	602	1,003	1,984	1.0×10^{-8}	3,750	6,019	0.220	pass
1342776288	593	1,003	1,986	1.2×10^{-8}	2,963	5,074	0.297	pass
8730574	598	1,003	2,005	8.3×10^{-9}	3,460	5,772	0.223	pass
1790307490	597	1,003	1,891	1.7×10^{-8}	2,590	4,825	0.250	pass
2126574302	592	1,003	1,956	6.8×10^{-9}	2,957	5,074	0.191	pass

All five jobs completed, and every saved check is within 1.7×10^{-5} of the original budget. Each run met the training gate after 592–602 active seconds. Each passed physical acceptance at its first inspection, after about 1,003 active training seconds and 1,891–2,005 SSBroyden iterations (Table 26). At that point stratified pressure is 0.19–0.30 of its limit, the tightest test in every run, stratified velocity 7.3–9.6 $\times 10^{-5}$ in relative L2, and the trajectory endpoint 0.04–0.05 of its limit. All five independent audits pass. The registered outcome is a qualified cohort: 5 of 5, Wilson 95% interval 0.57–1. The median time to verified accuracy is 5,074 seconds (84.6 minutes), with a range of 4,825–6,019 seconds on one GH200. Of the median run's time, about 1,006 seconds are training. The physical observation itself takes 2,612–3,757 seconds across runs, including the host spill of the

inverse Hessian, and imports, compilation and the new-seed checks take 1,091–1,297. Training to the gate therefore takes about a tenth of the time to verified accuracy; the complete native-AD physical observation dominates it. The TTA is the latency of confirmation under the registered inspection policy. The first physical crossing lies somewhere before the first inspection and is not measured. There is no paired baseline, so this cohort supports an absolute time to verified accuracy for this instance and policy, not a speedup. The five jobs used 7.48 allocated GPU-hours.

6.15 Paired engines to verified accuracy

A third registration supplies the baseline: the parameter-AD engine, the internal control of Sections 6.7–6.11, on five further new seeds. Each seed runs both engines one after the other on the same GH200 in one job. The order alternates across seeds, so both arms see the same device and node. Each arm stops at the first accepted SSBroyden iterate with a loss of at most 10^{-8}, and its threshold checkpoint is then observed once. The threshold is informed by development data: the nine audited passes above lie at losses of 6.7×10^{-9}–1.7×10^{-8}. The primary ratio is the parameter-AD arm's active training seconds to the threshold over the analytic arm's, recorded before any check of the threshold iterate. A pair counts only if both arms reach the threshold, pass physical acceptance and pass the independent audit. The registered rule calls the analytic engine faster if at least four such pairs give a geometric mean ratio above 1.03, with at least four ratios above 1.

A qualification run of both engines on development seed 361901 preceded the registration. The parameter-AD engine's eager, compiled and graph objectives agree with dense parameter AD within 8.7×10^{-7} of the original budget. With the dense inverse Hessian resident, both engines peak at 20.9 GB. Per evaluation, parameter AD takes 1.25 times as long in the Adam phase and 1.21 times in the SSBroyden phase. From these figures the registered prediction for the primary ratio is 1.23 (1.18–1.28).

Table 27: Paired C36 engine cohort (five new seeds, both engines on one GH200 per seed). Seconds are active training seconds to the loss threshold 10^{-8}; evaluations count the SSBroyden phase after the 4,000 Adam updates; S press. is the stratified corrected pressure of the threshold checkpoint over its limit. Ratio is parameter AD over analytic. Every arm passes physical acceptance and every audit passes.

	Analytic			Parameter AD			
Seed	Seconds	Eval.	S press.	Seconds	Eval.	S press.	Ratio
1344954027	1,025	3,104	0.235	1,280	3,101	0.220	1.249
164034124	1,010	3,015	0.248	1,262	2,976	0.242	1.250
2110545488	967	2,743	0.284	1,210	2,759	0.289	1.252
911819800	991	2,977	0.225	1,254	3,005	0.255	1.266
1176150139	1,028	3,245	0.235	1,305	3,259	0.225	1.269

All ten arms reach the threshold, pass physical acceptance and pass the independent audit (Table 27). At the threshold, stratified pressure is 0.22–0.29 of its limit in both engines. The analytic engine reaches the threshold after 967–1,028 active seconds and the parameter-AD engine after 1,210–1,305. The registered outcome is *analytic faster*: all five ratios favour the analytic engine, with a geometric mean of 1.257 (range 1.249–1.269), inside the predicted 1.18–1.28. The two engines need nearly the same number of evaluations (ratio 0.99–1.01), so the gain comes from cheaper evaluations: parameter AD takes 1.26–1.28 times as long per Adam evaluation and 1.24–1.25

times per SSBroyden evaluation, which includes the engine-independent optimizer work. This is the first solve-level C36 comparison with verified physical accuracy. It is smaller than the 1.32 gradient-throughput gain measured on H100 (Section 6.3), because the dense SSBroyden update and the line search add work that both engines share. The physical observation is excluded from the ratio, because it is the same native-AD observer in both arms. Including it would pull the ratio toward 1: one observation takes 2,673–3,813 seconds, about three times the training. The five jobs used 14.19 allocated GPU-hours.

6.16 External derivative backends

The paired cohort compares the analytic engine with an internal control. A further registration asks how public automatic-differentiation tools fare on the same objective. Three external backends produce the same 36-channel network state: nested PyTorch autograd over the batch, as an ordinary PINN code writes it; `torch.func` with `jacfwd` nested to third order plus the Hessian of the spatial Laplacian, vectorized with `vmap`; and the collapsed Taylor-mode package `jet` [11] at a pinned commit. The jet adapter uses collapsed Taylor mode for the value, the first derivatives and the spatial Laplacian, and autograd for the rest, as the Ethier–Steinman and two-dimensional adapters do; no Taylor producer of order three or more is implemented. The prepared geometry features are affine in space and the state needs one time derivative. A per-point polynomial built from the prepared geometry state therefore has exactly the derivatives the state uses, and every backend differentiates that same function. Gate, curl, consumer and loss are the C36 code in every method, and every external parameter gradient is autograd. On six training points each backend reproduces the analytic state within 7.2×10^{-4} of the original budget and the complete parameter gradient within 0.09.

Table 28: C36 objective and complete parameter gradient on one GH200 (seconds per evaluation; median window time, two stored initializations). Graph is a CUDA Graph replay of the compiled (analytic) or traced (external) evaluation. Ratios are against the analytic engine in the same mode (eager) and against its fastest variant (graph). Post-hoc rerun; see text.

Method	Eager (s)	Fastest (s)	Eager ratio	Fastest ratio
Analytic (C36)	1.71–1.74	0.134 (graph)	1	1
`torch.func`	1.71–1.72	0.496 (graph)	0.99	3.72
Collapsed `jet` + autograd	65.0–66.2	eager only	38.0	491
Nested autograd	430–435	eager only	251	3,242

The registration selects each method's fastest variant on a development initialization (eager, compiled or traced, and the CUDA Graph of either). Each variant is checked against dense parameter AD, also after an in-place parameter change, and then timed on the two stored initializations. The registered run did not complete. Its driver dropped the analytic CUDA Graph after a failed changed-parameter check, and the job then ran out of host memory while tracing the jet adapter, before nested AD and the timing phase. A diagnostic traced the graph failure to the driver. It returned only the replay closure, so the engine owning the geometry the graph reads was garbage-collected, and the next evaluation between two replays overwrote that memory. The trainers keep their engine alive and never showed this. A post-hoc rerun (r3) keeps every captured object alive and times the jet adapter eager only; everything else follows the registration. Every variant in it passes the dense-AD checks (Table 28).

In eager mode, `torch.func` matches the analytic engine (0.99, as predicted). The analytic advantage over it is therefore execution, not representation. Replayed as a CUDA Graph, the

analytic evaluation is 3.72 times faster than the fastest `torch.func` variant. The jet adapter and nested autograd are 38 and 251 times slower in eager mode, well above the registered predictions of 8.6 and 91. In both, nested autograd for the third and fourth spatial derivatives dominates; the eager analytic evaluation in this run (1.7 seconds) was also faster than in development (3.8). Against the analytic graph they are 491 and 3,242 times slower. By the registered feasibility rule, a full solve with `torch.func` (about 3.1 hours with observations) fits one job, while the jet adapter and nested autograd would need about 5.5 and 35 days. The three runs used 7.17 allocated GPU-hours.

6.17 Paired engines against `torch.func`

The feasibility rule above allowed one more registered cohort: the analytic engine against `torch.func` on whole solves, with the design of Section 6.15 unchanged. Five further new seeds run both arms one after the other on one GH200, in alternating order, as CUDA Graph replays to a training loss of 10^{-8}; each threshold checkpoint is observed once and every archive is re-read by the independent CPU auditor, extended for an arm without a compiled stage. The `torch.func` arm captures its eager evaluation at chunk 1,024: its `make_fx`-traced form held about 92 GB in the graph pool and twice failed the trainer's memory check before the registration (both failures are archived). The registered prediction for the ratio of `torch.func` to analytic active training seconds was 3.83 (3.6–4.1), from a qualification on the development seed: 3.98 per Adam evaluation and 3.65 per SSBroyden evaluation, weighted by the expected evaluation counts.

Table 29: Paired C36 cohort against `torch.func` (five new seeds, both arms on one GH200 per seed). Columns as in Table 27; Ratio is `torch.func` over analytic. Every arm passes physical acceptance and every audit passes.

	Analytic			`torch.func`			
Seed	Seconds	Eval.	S press.	Seconds	Eval.	S press.	Ratio
232943677	981	2,956	0.257	3,878	2,924	0.230	3.953
1952762959	1,006	3,007	0.235	3,919	2,988	0.236	3.895
1212253177	1,010	2,986	0.231	3,912	2,982	0.250	3.873
190009641	996	2,998	0.217	3,921	3,000	0.217	3.936
101379141	958	2,774	0.229	3,797	2,772	0.213	3.962

All ten arms reach the threshold, pass physical acceptance and pass the audit (Table 29); stratified pressure is 0.21–0.26 of its limit at the threshold in both arms. The analytic engine needs 958–1,010 active seconds and `torch.func` 3,797–3,921. The registered outcome is *analytic faster*: all five ratios favour the analytic engine, with a geometric mean of 3.92 (range 3.87–3.96), inside the predicted 3.6–4.1. The evaluation counts agree (0.99–1.00), so the whole gain is per evaluation: `torch.func` takes 4.0–4.1 times as long per Adam evaluation and 3.7–3.8 times per SSBroyden evaluation, the latter including the shared dense update. Both arms peak at 20.9 GB. This is the solve-level counterpart of the 3.72 graph-replay ratio of Table 28, and it closes the comparison that section left open. A development profile taken after the registration, not part of the cohort, found that the analytic graph replay is 3.17 times faster at chunk 2,048 than at the protocol's chunk 128 (0.042 against 0.132 seconds per evaluation), while the `torch.func` arm already runs near its best chunk, so the registered ratio understates the analytic advantage at its best chunk. The observation (2,661–4,048 seconds per arm) is excluded as before. The five jobs used 17.76 allocated GPU-hours.

7 Accepted three-dimensional solves: Ethier–Steinman

7.1 Benchmark, contract and baselines

The C36 study qualifies derivatives and throughput, and its solves pass only with a changed optimizer, against an internal parameter-AD baseline. We therefore also train on the transient three-dimensional Ethier–Steinman flow [4], which NSFnets also uses as a PINN benchmark [3]. With $a = 0.5$, $d = 1$ and $\nu = 1$,

$$u_* = -a\,[e^{ax}\sin(ay + dz) + e^{az}\cos(ax + dy)]\,e^{-\nu d^2 t}, \tag{21}$$

v_* and w_* follow by cyclic permutation of (x, y, z), the force is zero, and the pressure of this Beltrami flow is $p_* = -|u_*|^2/2$ up to a function of time. We use the cube $[-0.5, 0.5]^3$, $t \in [0, 0.25]$ and the gauge $p(x, t) - p(0, 0, 0, t)$. At four points, evaluated at 80 and 160 decimal digits, this field satisfies all 16 momentum, divergence and residual-gradient observations to below 10^{-65}.

A 4–64–64–4 tanh network with 4,740 parameters predicts (u, v, w, p) directly. Let r collect the three momentum residuals and the divergence. The FP64 objective is

$$L = \mathrm{mean}_\Omega\left(\|r\|_2^2 + 0.1\|\nabla_x r\|_F^2\right) + 10\,\mathrm{mean}_{\partial\Omega}\,\|u - u_*\|_2^2 + 10\,\mathrm{mean}_{t=0}\,\|u - u_*\|_2^2 + \mathrm{mean}_{\mathrm{gauge}}\,p^2 \tag{22}$$

over 512 interior, 384 boundary, 256 initial and 64 gauge points, where ∇_x is the spatial gradient. The residual reads $\partial_t u$, ∇u, Δu and ∇p; its spatial gradient additionally reads $\nabla\partial_t u$, the full spatial Hessian (through convection and the divergence gradient) and $\nabla\Delta u$. The C17 contract keeps exactly these: value, spatial gradient, packed spatial Hessian, gradient of the spatial Laplacian, $\partial_t z$ and $\nabla\partial_t z$, that is $1 + 3 + 6 + 3 + 1 + 3 = 17$ channels. Through an activation the spatial blocks follow Eq. (9), $\partial_t y = a_1\partial_t z$ and $\nabla\partial_t y = a_1\nabla\partial_t z + a_2\,\partial_t z\,\nabla z$; diffusion excludes time. Its explicit parameter transpose again uses a_4.

Two internal baselines share the data, initial parameter bytes, objective, optimizer program, FP64 CUDA Graph replay and gates. **A24** propagates the complete spatial third-order jet plus $\partial_t z$ and $\nabla\partial_t z$ (24 channels) and takes its parameter gradient by autograd. **Naive Q35** propagates all 35 monomials of (x, y, z, t) through degree three with a generic autograd parameter gradient. Both are internal implementations, not public libraries.

7.2 Protocol

Eight gates on separate validation sets of the same sizes must hold: normalized momentum, divergence, boundary-velocity, initial-velocity and gauge RMS at most 0.01, normalized residual-gradient RMS at most 0.02, velocity relative L_2 at most 0.02 and pressure relative L_2 at most 0.04. A candidate stop must be confirmed by independent scalar coordinate AD. Adam runs 2,000 updates with learning rate 10^{-3}; strong-Wolfe L-BFGS then permits at most 400 blocks of at most 20 iterations, nominally 25 evaluations, history 50 and tolerances $10^{-10}/10^{-12}$. Stop checks run every 200 Adam updates and every five blocks. Before training, each process checks the complete loss and all 4,740 gradients against scalar AD on a small point subset and, through three in-place graph replays on all points, against the eager A24 reference at absolute 2×10^{-12} and relative 2×10^{-11}.

A development protocol with 100 L-BFGS blocks, run on one H20 with seed 642201, passed seven gates in all three arms and failed only pressure relative L_2 (6.95%, 6.95% and 7.04%). Removing each model's own pressure offset at the origin did not change that error, and it was still decreasing at the final block. The successor protocol changes only the block budget, to 400; a CPU development run of C17 on the same seed then passed every gate. That seed is excluded. The formal protocol

fixes six seeds, 642202–642207, before any formal run; four are new draws from the original initializer, which reproduces the earlier three bit for bit.

All 18 cells run once on two NVIDIA H100 GPUs of one node with Torch 2.4.0a0 and NumPy 1.26.4, FP64 and TF32 disabled. The three arms of a seed share a GPU, and the six seeds use the six arm orders, so every arm occupies every position twice. Each case is a fresh process; nothing is retried. Times are setup and graph capture plus optimization, including stop checks and the confirming independent check; the pre-training qualification and the archived final field evaluation are reported separately. An independent NumPy reader rechecks every saved qualification gradient, the final fields against a separate 35-monomial Taylor oracle, all metrics and every acceptance decision.

7.3 Results

Table 30: Ethier–Steinman solves on H100. Accepted means both validators pass all eight gates. Times are medians of setup plus optimization in seconds. Evaluations are complete loss/gradient evaluations, including Adam. The ratio is the geometric mean of per-seed baseline/C17 ratios over n jointly accepted pairs; values above one favor C17.

Arm	Accepted	Setup + opt. s	Evaluations	Ratio / n	Range
FlashNS C17	6/6	55.7	6,676	1.000	–
A24 reference	6/6	210.1	7,220	3.602 / 6	3.353–4.284
Naive Q35	6/6	386.6	6,675	6.957 / 6	6.750–7.436

All 18 formal solves pass both validators (Table 30). C17 reaches the gates 3.602× faster than A24 (six accepted pairs, range 3.353–4.284) and 6.957× faster than naive Q35 (range 6.750–7.436). The arms need similar numbers of evaluations, with acceptance between L-BFGS blocks 165 and 270, so the gain comes from cheaper complete loss/gradient evaluations rather than from fewer optimizer steps. Measured as setup-plus-optimization time per evaluation, A24 and naive Q35 cost 3.537× (3.311–3.713) and 7.034× (6.757–7.300) as much as C17, which spends 8.0–8.7 ms per evaluation including optimizer and stop-check work. These ratios measure the contracted state and the explicit parameter transpose together, because both baselines also use autograd for the parameter gradient; neither change is isolated here. As v in two dimensions, pressure relative L_2 is the binding gate: its largest final values are 0.03966 (C17), 0.03972 (A24) and 0.039999 (naive Q35) against 0.04, while every other gate stays at or below 22% of its limit. The independent reader verifies 14 source and 91 raw files and passes 1,512 comparisons over 2,227,032 numbers, with maximum absolute discrepancy 1.78×10^{-14}.

These are accepted three-dimensional solve comparisons against internal baselines on a small network (4,740 parameters, 512 interior points). C17 is a velocity–pressure contract without a divergence-free construction; the result says nothing about C36 training. The two GPUs ran concurrently on a node whose other GPUs carried another workload, so absolute times include that environment. The paired ratios compare arms of one seed on one GPU.

7.4 External derivative baselines

The review asked for an external baseline on the three-dimensional C36 objective. C36 has no accepted solve, so a later registered batch answers that request on this benchmark instead; the C36 request stays open. Its protocol was committed before the batch ran. Two public derivative methods compute the unchanged objective without FlashNS code:

- **Nested AD**, nested scalar-coordinate PyTorch autograd as an ordinary PINN code writes it: four first-derivative passes, nine for the three spatial Laplacians and one per residual for its spatial gradient;
- **collapsed jet**, the public collapsed Taylor-mode package of Dangel et al. (`jet`, pinned snapshot) for the spatial first derivatives and the spatial Laplacian, with a first-order jet for ∂_t.

Both arms take the residual gradient and the parameter gradient by autograd, as the adapters of the two-dimensional comparison do. Each arm, including C17, replays its complete loss and gradient as a CUDA Graph, and no arm is compiled; the two-dimensional comparison instead let every baseline run its faster qualified default or compiled variant. All 18 solves run once in one batch on one four-GPU H100 node with Torch 2.12. Data, seeds, initial parameter bytes, objective, optimizer, stop checks, qualification and acceptance are those of the formal protocol. Before the batch, both external arms matched independent scalar AD on CPU within the qualification budget.

Table 31: External derivative baselines on the Ethier–Steinman solves (one H100 batch, Torch 2.12). Columns as in Table 30. Adam and L-BFGS are the post-hoc time per evaluation in each optimizer phase, including that phase's optimizer step and stop checks.

Arm	Accepted	Setup + opt. s	Evaluations	Ratio (range)	Adam ms	L-BFGS ms
FlashNS C17	6/6	47.2	6,728	1.000	2.4–2.7	8.4–9.3
Collapsed jet	6/6	52.6	6,610	1.085 (0.98–1.19)	3.6–3.7	9.0–9.8
Nested AD	6/6	89.0	6,709	1.901 (1.67–2.14)	8.3–10.1	14.2–15.5

All 18 solves pass both validators (Table 31). Pressure relative L_2 again binds, with largest values 0.03994, 0.03997 and 0.03983 against 0.04; every other gate stays at or below 22% of its limit. The independent reader passes 1,512 comparisons with maximum absolute discrepancy 1.78×10^{-14}.

In setup plus optimization, the registered statistic, C17 is 1.90 times faster than nested autograd and 1.08 times faster than the collapsed jet. Optimization alone gives 1.90 and 1.08, and time per evaluation 1.91 and 1.10.

A post-hoc split by optimizer phase shows why the collapsed jet comes so close. In the Adam phase, where the gradient dominates each evaluation, the collapsed jet costs 1.37 times and nested autograd 3.62 times as much as C17 per evaluation. Each L-BFGS evaluation adds about 6 ms in every arm, as in two dimensions (Section 5.6), which compresses the solve ratios to 1.07 and 1.70 in that phase. Against a strong public Taylor-mode implementation, the contracted state and explicit transpose therefore shorten this small problem's solve by only about 8%, although an Adam-phase evaluation, which contains the gradient, is 1.37 times faster. One of the six seeds favours the collapsed jet, and the protocol registered no gain rule, so the 1.08 is a measured ratio rather than a decided gain. Larger problems, where the gradient dominates, were not run.

Compiled variants. In the two-dimensional campaign on H100 (Section 5.8), compiling made the collapsed-Taylor, ED and torch-AD replays 3.2, 5.0 and 2.3 times faster. A second registered batch therefore runs every method in two variants, graph-only and compiled. The compiled variant traces the whole loss and gradient with `make_fx`, compiles it with `torch.compile` and captures it in one CUDA Graph. Everything else is unchanged from the batch above, and the 36 solves run in one batch. Each lane first runs every arm on a development seed without training, and each method's faster qualified variant on that warm-up replay is selected before any formal solve is read. The registered statistic is the selected external variant over the selected C17 variant, in setup plus

optimization (primary) and in optimization alone (secondary), with the rule of the solve-scale study (Section 5.6): a ratio beyond 1.03 with at least five of six favourable pairs.

Table 32: Compiled variants on the Ethier–Steinman solves (one H100 batch). Warm-up replay is the median over four lanes on the development seed. Medians are over the six formal seeds. Adam is the post-hoc time per evaluation in the Adam phase. Bold marks the variant selected for each method.

Method	Variant	Accepted	Replay ms	Setup + opt. s	Opt. s	Adam ms
C17	**graph**	6/6	2.52	46.3	46.2	2.3–2.7
	compiled	failed qualification (CUDA Graph capture), 10 of 10 attempts				
Collapsed jet	graph	6/6	3.45	50.2	50.0	3.6
	compiled	6/6	0.85	49.1	38.3	0.9–1.0
Nested AD	graph	6/6	9.76	88.1	87.8	8.8–10.0
	compiled	6/6	2.39	73.0	46.4	2.5–2.6

The compiled C17 variant failed in all ten attempts. Its warm-up calls succeeded, but recompilation during CUDA Graph capture hit a generator that the compiler cannot trace; the cause was not found. C17 therefore runs as graph-only, and both external methods run compiled (Table 32). All 30 solves of the five qualified arms are accepted.

Against compiled nested autograd, C17 is 1.59 times faster in setup plus optimization (1.50–1.75, six of six pairs), the registered "C17 faster". In optimization alone they are level (1.01). Against compiled collapsed jet, setup plus optimization gives 1.05 (0.95–1.24, three of six pairs), so there is no decided difference. In optimization alone the collapsed jet is faster: it needs 0.81 times C17's time (0.74–0.99), with all six pairs favouring it, the registered "external faster". Its compile adds 10.6–11.1 s of setup, which absorbs the gain in setup plus optimization. These compiled setups are below the warm-up setups, so the registered cache caveat does not apply. The graph-only arms reproduce the batch above: 1.11 for the collapsed jet and 1.92 for nested autograd.

A post-hoc phase split locates the difference. In the Adam phase the compiled collapsed jet costs 0.39 times as much per evaluation as graph-only C17, so it is about 2.6 times faster, while compiled nested autograd is level (1.01). The fixed L-BFGS cost then compresses the solve ratio, as elsewhere. On this benchmark, a public Taylor-mode package compiled by `torch.compile` therefore evaluates the gradient faster than the C17 contract in eager PyTorch. Unlike two-dimensional GL8, whose native pointwise kernels the ablation credits with 1.53–1.83 (Section 5.6), C17 has no execution layer.

Compiled C17, second attempt. A one-GPU diagnostic reproduced the capture failure for both `make_fx` routes. Compiling a function that takes the six parameter tensors as explicit arguments, the pattern of the two-dimensional engine, survived capture and matched eager C17. A second registered attempt changes only the compiled C17 builder to this route; arms, data, seeds, warm-up, selection and decision rules are those of the first. All 36 solves pass, and every method selects its compiled variant (Table 33).

With every method compiled, C17 is 1.68 times faster than nested autograd in setup plus optimization (1.44–1.90, six of six pairs) and 1.17 times in optimization alone (five of six), both the registered "C17 faster". Against the collapsed jet, setup plus optimization gives 1.13 (0.99–1.20, five of six), again "C17 faster". Optimization alone gives 0.96 (0.85–1.01, five of six favouring the jet), the registered "external faster".

The two decisions differ because of setup. Compiled C17 takes 3.5–4.0 s, because its explicit-argument builder reuses the compile cache across seeds. The collapsed jet's builder captures the

Table 33: Second compiled attempt on the Ethier–Steinman solves (one H100 batch, all 36 solves accepted). Replay is the median warm-up replay over four lanes. The other columns are for the selected, compiled variant: medians over the six formal seeds, setup range, and the post-hoc Adam-phase time per evaluation.

Method	Graph ms	Compiled ms	Setup + opt. s	Opt. s	Setup s	Adam ms
C17	2.55	1.20	43.9	40.2	3.5–4.0	1.15–1.34
Collapsed jet	3.48	0.86	49.6	38.9	10.7–10.9	0.99–1.00
Nested AD	8.54	2.41	71.9	45.5	25.6–26.5	2.32–2.56

parameters as constants and recompiles part of its graph in every solve (10.7–10.9 s). Per evaluation the compiled jet is 5% faster, and its Adam-phase evaluation 1.3 times faster. Compilation makes C17's replay 2.1 times faster than graph-only C17. With all methods compiled, the contracted state therefore gives no per-evaluation advantage over collapsed Taylor mode on this small benchmark, and C17's whole-solve lead over it rests on cheaper setup.

8 Discussion and limitations

Attribution of the two-dimensional gain. The derivative consumer offers an execution opportunity even after replacing nested coordinate AD with Taylor-mode computation. GL8 removes independent third-order information that the fixed diffusion-gradient consumer never reads, while preserving lower derivatives required by nonlinear convection. The H20 measurement combines representation, pointwise kernels, graph scheduling and parameter backward.

The H100 ablation of Section 5.6 separates them for this problem. With the execution held fixed, the GL8 state alone is 1.09 times faster than Q10 at the original 2,048 points; this is unresolved, because the A/A control failed at that size. It is 1.63–1.66 times faster from 16,384 points on. Native pointwise kernels add 1.53–1.83 at every size, and the GEMMs are identical, so that factor is pure pointwise execution. At the size of the H20 campaign, most of the full-gradient gain therefore comes from execution, as the earlier profile suggested; the state matters at larger point counts. All of this is one problem on one GPU family; no native Q10 kernel was available, and wider networks and other PDEs were not tested beyond four topologies.

Replay versus solve. Replay and solve ratios need not match. Optimizer work, line searches and fixed validation remain outside the accelerated derivative program. Small floating-point differences can change an L-BFGS trajectory and its closure count. Setup also differs between compiled public adapters, JAX and explicit native kernels, even with warmed dependency caches. We therefore report every timing scope and actual evaluation count rather than extrapolating a replay factor to the whole training run. On H100 this matters decisively. Each L-BFGS evaluation costs 5.9–7.1 milliseconds outside the gradient at every measured size. At 2,048 points, where the native GL8 gradient replays in 0.17 milliseconds and compiled Q10 in 0.28, optimization alone is no faster (0.95). At 16,384 and 65,536 points the same cost is a smaller share of each evaluation, and native GL8 shortens optimization by 1.48 and 1.88 under a registered rule. Whether the solve-level gain appears therefore depends on problem size and on the optimizer's own cost, not on the derivative program alone. Native setup avoids compilation, but at larger sizes queue order decided which solves met a cold compiler cache, so the decisions use optimization time alone.

Chunking. Chunking addresses a different engineering constraint. It bounds the intermediate execution workspace and permits a substantial allocator reduction at the measured larger shape, while paying for input copies, extra replay launches and gradient accumulation. The fixed data and optimizer remain resident. We measured neither an OOM boundary nor the maximum trainable point count, and Torch allocator counters cannot rank total process memory against JAX's allocator.

Scope of the external comparisons. ED and collapsed Taylor supply public second-order operators extended to a third-order gradient-enhanced consumer; this is not their original training benchmark. Exact two-coordinate STDE preserves a deterministic target but does not test stochastic amortization in high dimensions. The unmodified cuEquivariance polynomial arm has an all-points Q10 boundary policy, unlike the compact interior/value-only policy of the paper adapters. The six initializations provide a paired view of one fixed-data manufactured PDE, not variation across PDEs, sampled datasets, network families or hardware. No speed claim is extended to stochastic STDE++ constructions, variable viscosity, nonlinear coordinate maps, sample-coupled layers, second parameter derivatives or nonlocal losses. Reduced precision is measured only for two 2D cells (Section 5.7); every other measurement uses FP64.

Three dimensions. The Ethier–Steinman solves establish an accepted three-dimensional solve-level gain for a 17-channel transient contract on a small network: 3.60 and 6.96 against internal baselines. With every method compiled, it is 1.68 times faster than nested autograd and 1.13 times faster than public collapsed Taylor mode in whole solves. The latter, however, is 5% faster per evaluation, so C17's lead over it rests on cheaper setup. In a first attempt C17's compiled variant failed to capture, and graph-only C17 lost to the compiled collapsed jet in optimization by 1.23 times. Unlike two-dimensional GL8, C17 has no native execution layer. The C36 study establishes derivative correctness for the full objective of a larger curl-potential model and a throughput gain for its gradient against an internal control, but no accepted solve. With 800 L-BFGS calls its training passes the loss gate, the uniform holdout and the initial and boundary tests, while stratified pressure stays 2–3 times above its limit inside the moving box and the trajectory endpoint is borderline. Seventy-two observations along the training trajectories show that error still falling at that budget, more slowly than the loss. With six times the L-BFGS budget the loss and the errors stall, so the gap is not a budget limit of this optimizer. On an instance with twice the cutoff width, whose layer forcing is four to five times smaller, stratified pressure still ends 2.0–2.9 times above its limit, so the gap is not set by the layer forcing alone either. A temporal-smoothness continuation changes where training meets the loss gate but not which test fails. The optimizer does: a dense self-scaled Broyden phase after the same Adam updates passes every physical test in two development runs on GH200, with stratified pressure about 10^{-3} of its limit after 4,000 training seconds. A first registered five-seed cohort is not qualified: its frozen auditor's relative tolerance rejects ARM–x86 rounding in a cancelled sum, and one run was cancelled. A second cohort with a rounding-bounded auditor passes 5 of 5, with a median of 5,074 seconds to verified accuracy, mostly spent in the physical observation. In a registered paired cohort the analytic engine reaches the verified loss threshold 1.257 times sooner than the parameter-AD control (1.249–1.269, 5 of 5 pairs). The two engines need the same number of evaluations, so the gain is per evaluation; dense SSBroyden work shared by both engines keeps it below the 1.32 gradient-throughput gain. The dense inverse Hessian costs $8n^2$ bytes (20.6 GB here), so on one 96 GB GPU this optimizer does not reach beyond about 10^5 parameters. Against public tools on the same objective, per evaluation, `torch.func` equals the analytic engine in eager mode and is 3.72 times slower as a CUDA Graph replay, so the C36 advantage over it is execution rather than representation, and a second registered cohort carries

this to whole solves: 3.92 times sooner to the verified threshold in all five pairs (Section 6.17); collapsed `jet` with autograd for the higher orders, and nested autograd, are 38 and 251 times slower in eager mode. These are post-hoc measurements after a driver bug and a host-memory failure ended the registered run. The C36 throughput gains depend on compilation, the zero-gate folding gain depends on this instance's gate geometry, and the problem is a single constructed box instance. A solve-level comparison with `torch.func`, a higher-order Taylor-mode producer for the jet adapter, a native C17, a second nonlinear PDE and device-traffic measurements remain open experiments rather than implied consequences of the implementation.

9 Conclusion

FlashNS gives a concrete consumer contract for high-order PINN training: propagate a closed value/gradient/Hessian/gradient-Laplacian state and transpose every local dependency, including the fourth activation derivative, into the complete parameter gradient. Under an unchanged FP64 Navier–Stokes objective with independent acceptance, all 72 planned two-dimensional H20 solves pass and GL8 shortens the paired solve time by 1.66× over a compiled full-jet control and 1.73–3.37× over matched public derivative methods; on H100 the same campaign passes all 78 solves with these ratios compressed to 1.22× and 1.29–1.69×; a chunked variant trades replay time for an 85% lower allocated setup peak at 16,384 points. On H100, a registered ablation attributes the full-gradient gain mainly to native pointwise execution at that size (1.57) and increasingly to the state at larger point counts (1.63–1.66 from 16,384 points on). In whole solves a fixed L-BFGS cost of about 6–7 milliseconds per evaluation absorbs the gain at the original size; a registered follow-up finds solve-level gains of 1.48 and 1.88 at 16,384 and 65,536 points. That fixed cost is the optimizer's host-side execution: on GH200 a device-side two-loop leaves all 36 solves bitwise unchanged and shortens optimization 2.21 times at 2,048 points (Section 5.9; the timing flag failed on one drift repeat). FP32 training passes the unchanged FP64 acceptance and shortens 65,536-point optimization by 1.36–1.53× at these 1–2% gates, while every TF32 solve fails acceptance. On the transient three-dimensional Ethier–Steinman benchmark, a 17-channel contract passes all gates in 18 of 18 H100 solves and is 3.60× and 6.96× faster than internal full spatial-jet and naive baselines; with all methods compiled, its solves are 1.68× and 1.13× faster than nested autograd and public collapsed Taylor mode, although the latter is 5% faster per evaluation and C17's lead rests on cheaper setup. The C36 curl-potential extension passes complete gradient qualification and improves objective-gradient throughput by 1.32× over an equally compiled parameter-AD control, and by 1.40× when both arms fold exact zero-gate rows. Its original training budget does not reach the training gate; a development run with 40 times more L-BFGS calls does, and passes most physical gates, but stratified pressure still fails. Observations of every checkpoint show this error still falling at that budget. With six times the L-BFGS calls it stalls at 2.2–3.0 times its limit; neither a smoother-motion continuation nor an instance with twice the cutoff width (2.0–2.9 times) removes it. A dense self-scaled Broyden optimizer does: two development runs on GH200 pass every physical test. A first registered five-seed cohort is not qualified, because its frozen auditor rejects cross-architecture rounding. A second cohort, with a rounding-bounded auditor, passes 5 of 5 with a median of 5,074 seconds to verified accuracy on one GH200. In a paired cohort the analytic engine reaches the verified loss threshold 1.26× sooner than the parameter-AD control in all five pairs. Per evaluation, `torch.func` equals the analytic engine in eager mode and is 3.72× slower as a CUDA Graph replay, and in a second paired cohort the analytic engine reaches the verified threshold 3.92× sooner than `torch.func` in all five pairs; a collapsed-`jet` adapter and nested autograd are 38× and 251× slower in eager mode.

References

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. https://doi.org/10.1016/j.jcp.2018.10.045.

[2] Jeremy Yu, Lu Lu, Xuhui Meng, and George Em Karniadakis. Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. arXiv:2111.02801, 2021. https://arxiv.org/abs/2111.02801.

[3] Xiaowei Jin, Shengze Cai, Hui Li, and George Em Karniadakis. NSFnets (Navier-Stokes flow nets): Physics-informed neural networks for the incompressible Navier-Stokes equations. *Journal of Computational Physics*, 426:109951, 2021. https://doi.org/10.1016/j.jcp.2020.109951.

[4] C. Ross Ethier and D. A. Steinman. Exact fully 3D Navier–Stokes solutions for benchmarking. *International Journal for Numerical Methods in Fluids*, 19(5):369–375, 1994. https://doi.org/10.1002/fld.1650190502.

[5] Dezhen Lu. FlashNS Ethier–Steinman protocol, source and audited result. Frozen protocol v4, executed-source hashes and result JSON files.

[6] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Second edition, SIAM, 2008.

[7] Jesse Bettencourt, Matthew J. Johnson, and David Duvenaud. Taylor-mode automatic differentiation for higher-order derivatives in JAX. NeurIPS 2019 Workshop on Program Transformations, 2019. https://openreview.net/pdf/025f91ed7c1059e6cb35b1704fc887452a4084f8.pdf.

[8] Ruichen Li, Haotian Ye, Du Jiang, Xuelan Wen, Chuwei Wang, Zhe Li, Xiang Li, Di He, Ji Chen, Weiluo Ren, and Liwei Wang. A computational framework for neural network-based variational Monte Carlo with Forward Laplacian. *Nature Machine Intelligence*, 6(2):209–219, 2024. https://doi.org/10.1038/s42256-024-00794-x.

[9] Jack Richter-Powell, Yaron Lipman, and Ricky T. Q. Chen. Neural conservation laws: A divergence-free perspective. *Advances in Neural Information Processing Systems*, 35, 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/f8d39584f87944e5dbe46ec76f19e20a-Abstract.html.

[10] Wenbo Cao, Zhe Lu, and Weiwei Zhang. Verified residual-specific explicit derivative kernels for physics-informed learning and discretized PDE adjoints. arXiv:2606.29702, 2026. https://arxiv.org/abs/2606.29702.

[11] Felix Dangel, Tim Siebert, Marius Zeinhofer, and Andrea Walther. Collapsing Taylor Mode Automatic Differentiation. *Advances in Neural Information Processing Systems*, 38, 2025. https://papers.nips.cc/paper_files/paper/2025/hash/d835e25415cfe4d6d48a0be3b7b763bb-Abstract-Conference.html.

[12] Zekun Shi, Zheyuan Hu, Min Lin, and Kenji Kawaguchi. STDE++: Polynomial-Time Amortization for Linear Differential Operators. *Journal of Machine Learning Research*, 27(114):1–50, 2026. https://www.jmlr.org/papers/v27/25-1474.html.

[13] Zekun Shi, Zheyuan Hu, Min Lin, and Kenji Kawaguchi. Stochastic Taylor Derivative Estimator: Efficient amortization for arbitrary differential operators. *Advances in Neural Information Processing Systems*, 37, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/dd2eb5250696753ea37141bbd89bb569-Abstract-Conference.html.

[14] sail-sg. STDE official public implementation. Source commit `fae88663b1d2f1b0666d4d250c38a3c34b852ac0`. https://github.com/sail-sg/stde.

[15] NVIDIA. PhysicsNeMo public implementation. Source commit `b45a5c810c741e6b41f8515be24c51121f8fc21f`. https://github.com/NVIDIA/physicsnemo.

[16] NVIDIA. CUDA Deep Neural Network library (cuDNN). https://developer.nvidia.com/cuda/cuda-x-libraries/cudnn.

[17] NVIDIA. cuEquivariance public implementation. Source commit `70674ada7a00372e88ebe99969c700a6dcddd08f`. https://github.com/NVIDIA/cuEquivariance.

[18] Jorge F. Urbán, Petros Stefanou, and José A. Pons. Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be? *Journal of Computational Physics*, 523:113656, 2025. https://doi.org/10.1016/j.jcp.2024.113656.

[19] Elham Kiyani, Khemraj Shukla, Jorge F. Urbán, Jérôme Darbon, and George Em Karniadakis. Optimizing the optimizer for physics-informed neural networks and Kolmogorov–Arnold networks. arXiv:2501.16371, 2025. https://arxiv.org/abs/2501.16371.

[20] OpenAI. Incompressible Box Transport and Finite Computation. September 27, 2026. Mathematical geometry source pinned at commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. Original box-transport potential, curl extension and forced-flow realization.

[21] Dezhen Lu. FlashNS C36 experiment and reproducibility artifact. Evidence snapshot `37ced2805131fcfaba6f7f89f8b0b1d17e5cef28`, October 13, 2026; per-experiment runtime hashes are retained in the result files. Public C36 source, frozen protocols and result JSON files.

A Explicit local transpose

For completeness, let $\bar{y}$, $\bar{g}_y$, B_y and $\bar{t}_y$ denote output adjoints. Here B_y is symmetric and represents the Frobenius pairing with the full symmetric Hessian. If the stored off-diagonal Hessian channel has adjoint β_{12}, this convention sets $(B_y)_{12} = (B_y)_{21} = \beta_{12}/2$. Define $h = \operatorname{tr} H$, $c = g^\top g$ and $Q = gh + 2Hg$. Transposing the local differential gives

$$\bar{t} = a_1 \bar{t}_y, \tag{23}$$

$$\bar{H} = a_1 B_y + a_2((\bar{t}_y^\top g)I + \bar{t}_y g^\top + g\bar{t}_y^\top), \tag{24}$$

$$\bar{g} = a_1 \bar{g}_y + 2a_2 B_y g + a_2(h\bar{t}_y + 2H\bar{t}_y) + a_3(c\bar{t}_y + 2g(g^\top \bar{t}_y)), \tag{25}$$

$$\bar{z} = a_1 \bar{y} + a_2 \bar{g}_y^\top g + \langle B_y, a_2 H + a_3 g g^\top \rangle_F + \bar{t}_y^\top (a_2 t + a_3 Q + a_4 g c). \tag{26}$$

The packed input Hessian adjoint is obtained by transposing symmetric unpacking: diagonal entries are unchanged and each off-diagonal channel receives the sum of its two matrix entries. This convention prevents a factor-of-two error. All terms contribute to the shared parameter VJP

through the affine transpose; no activation derivative order is discarded because it exceeds the forward coordinate order.

The argument assumes fixed physical coordinates, coordinate-independent affine parameters, a fixed contraction and sufficiently smooth pointwise activations. Let P map a complete third-order raw jet to (z, g, H, t). The chain rule establishes $PF_{\text{full}} = F_{\text{GL8}}P$ for each supported layer. Induction gives the same identity for a supported network. Since the residual loss factors through P, its real-arithmetic parameter derivative agrees with the full-jet derivative. This is a closure and transpose argument for a stated consumer; it is not a proof that no other representation could use fewer channels.

B Chunk accumulation and storage contract

Partition interior and boundary points into finite chunks. With the original weights, the loss is the sum of chunk contributions and its parameter gradient is the sum of their parameter gradients. The executor zeros accumulators once at the beginning of a closure, copies a chunk into the reusable graph inputs, replays, and accumulates its loss and gradient. Tail padding has zero weight. Every chunk uses the same parameter version; the optimizer can update parameters only after the complete closure.

For maximum chunk size M, intermediate derivative storage is proportional to M times the retained layer-channel widths. Fixed point data still grow with total point count, and parameter/optimizer storage is not bounded by M. The eight-channel contraction removes two hidden coordinate channels; it does not by itself prove a 20% reduction in total live memory. At 2,048 interior points, compact logical point/channel rows decrease from $10(2048) + 512 = 20,992$ to $8(2048) + 512 = 16,896$. Actual allocator measurements include further temporaries and graph pools.

Captured outputs are borrowed until the next replay. Graph parameter storage can be updated in place; replacing its storage or using an incompatible stream/layout is rejected. Comparing a previously returned borrowed scalar after another closure would compare an overwritten value. The validation harness checks each live loss/gradient before replay and checks updated parameter storage separately.

C Artifact, provenance and execution records

C.1 Two-dimensional H20 campaign

The original two-dimensional H20 execution source is the commit

`286d7e394c8bd39f9aa409715afe9f1bac915cec`,

based on public main `7c261ea5635c` (full ID `7c261ea5635cdcec1bf98b72d1105dcf0b486087`) and a preserved local implementation snapshot. The baseline adapters are separate frozen runtime files. Table 34 identifies upstream sources; full tree/blob hashes, runtime file SHA256 values and all invocation arguments accompany the artifact. Pinning distinguishes the public collapsed transform from the project's own compiled full-jet backend.

The representation, memory and profiler study used physical H20 `edf64e5f`; the public comparison, with six planned seeds, uses physical H20 `d952768b`, as do the NVIDIA comparisons. Earlier public campaigns remain archived separately and are not pooled with the current comparison. Pairwise ratios are computed within their own device-specific campaign. Dependencies live in a

Table 34: Pinned public baseline sources. Full commit IDs are retained in the machine-readable source manifests.

Implementation	Public repository	Commit prefix
Cao ED	Cao-WenBo/ExplicitDifferentiation	`a524575baa62`
Collapsed Taylor	f-dangel/torch-jet	`2e49005ee4ae`
STDE	sail-sg/stde	`fae88663b1d2`
PhysicsNeMo	NVIDIA/physicsnemo	`b45a5c810c74`
cuEquivariance	NVIDIA/cuEquivariance	`70674ada7a00`

task-owned environment inheriting the unchanged base numerical stack. The shared H20 runs one admitted GPU window at a time; all recorded children finish naturally, and whole-window ownership includes copying and checking evidence before explicit handback.

The cuEquivariance polynomial adapter requires an initialization-only portability repair: its unused inherited FlashNS native-build initialization is omitted, while polynomial evaluation and all derivative and parameter-gradient functions remain inherited without modification. An initial recovery attempt fails at import before numerical evaluation and is retained with its process cost. The corrected recovery passes all three updated-parameter checks and the native-path profile before formal training. Scalar tensor-product variants fail higher-order autograd qualification; these failures characterize the tested route and do not establish a general limitation of cuEquivariance. The NVIDIA qualification retains four failed execution variants with complete tracebacks and process costs; no failed backend is renamed or replaced by the scalar diagnostic.

Original JSON reports, progress histories, complete profiler event names, stdout/stderr, final parameter tensors, protocols and natural process-exit receipts are preserved. Every archive is copied off the GPU instance and checked against its recorded whole-archive SHA256 before extraction; every manifested member is then checked individually. Analysis rejects altered data, optimizer configurations, unmatched initial parameters, repeated seed/arm attempts and claimed convergence that violates either validator.

One inherited optimizer-protocol description still lists the earlier seeds 75101–75103 in its field `formal_initialization_seeds`. That field does not select parameters in the solver. The frozen campaign seed list, each actual child's `-seed` argument and the reported initial tensor-byte hashes identify the six formal initializations 862101–862106. We preserve the original metadata bytes and state this discrepancy rather than silently rewriting the raw reports.

Preparation-only failures are also retained. A local boundary-cardinality assertion initially counted shared outlet coordinates twice; the frozen problem always had 512 boundary coordinates, and no numerical data changed. An earlier archive check correctly rejected an SCP transfer before transfer completion. Neither event caused a formal training retry or a changed scientific threshold. GPU comparison uses only the final frozen compact adapter packet, whose 460 runtime source hashes are checked before each admitted window.

The initial implementation regression ran 547 CPU checks with two existing upstream Euler dependency skips and 129 full-width GPU checks. Full-width Compute Sanitizer memcheck completed 112 checks with zero errors. Small network racecheck, initcheck and synccheck each completed 60 checks with zero errors; an earlier full-width racecheck was interrupted for instrumentation cost and is not counted as completed. These tests establish stated regression coverage, not correctness for every unsupported consumer.

C.2 Three-dimensional C36 study

The problem source and calibration hashes agree with the provenance recorded in the frozen training cache, including the original mathematical commit and source-file hash. The full native reference was evaluated on H100 with native coordinate-AD chunks of 256 points; candidate partitions of 128 and 64 points, nonzero value-family diagnostic probes and caller-data mutation checks also pass. Packed-gather source `ae51ca643280` retains the C36 algebra and local state-autograd pullbacks; its qualification records 48 comparisons and 96 saved arrays without rerunning the expensive native reference, and partition agreement alone is not an independent coordinate oracle.

The first compiled CUDA attempt retained 18 passing direct full-gradient checks but stopped before compiled evaluation because Triton was unavailable; the whole attempt remains failed and archived. The eager packed-graph experiment explicitly selects eager activation plus CUDA Graph replay and does not fall back from a compiler failure. The completed compiled experiment uses Torch 2.4.0a0 and Triton 3.0.0. That cluster runtime supplies compiler targets as tuples; an explicit per-process adapter changes only that representation to Triton's device target, checking backend, architecture and warp size against the actual device. Kernel source, FP64 dtype and compiler options are unchanged, both arms use the same adapter, and no installed Torch or Triton source is modified. The CPU variant uses Torch 2.14.1, NumPy 2.5.4, CPU Inductor and one Torch thread; a fresh NumPy-only reader checks 24 saved gradient arrays, 12 gradient comparisons and all 20 paired timing windows.

For the eager and compiled H100 throughput experiments, both workers exit zero; the complete archives (106 members, 161,241,343 bytes; 114 members, 161,322,508 bytes) are verified on cluster storage and locally, all owned children finish and the original GPU lock is released. The CPU packet is verified locally only.

A separate older source, `73baa3b281fb`, measures native coordinate AD with its original recursive derivative cache and the original native rule of one complete native evaluation per window. Mean candidate/native times are 2.777272/3,484.283522 seconds, with paired ratio 1,254.839 and interval 1,211.337–1,291.263 (ten pairs). That candidate predates packed gather, and the baseline predates the later cache-lifetime repair. The large ratio is specific to that slow recursive native path; it is not a speedup against the optimized parameter-AD control, the repaired native implementation or an accepted solver. The timing worker, reader and timing driver exit zero. The original whole controller exits one due to its earlier failed training stage, which remains a failed controller outcome.

The training sources are `7281faed9ec4` (explicit) and `de1d223` (parameter AD). Complete training histories have verified cluster archives, while smaller training anchors are also available locally. The physical observer source `1f1983a2b015` repairs a recursive native derivative closure cycle that retained CUDA graphs until garbage collection; the earlier out-of-memory failure and its raw logs remain archived. The repair preserves derivative order and parameter dependence. It is checked against original native states and all 50,692 gradient entries at two points; sixteen repeated native observations keep allocated CUDA memory constant at 67,514,880 bytes. Each checkpoint's remote and local NumPy readers pass 117 saved-array checks. They verify hashes, loss/gradient anchors, field residual metrics, trajectory error summaries and refinement statistics; they do not rerun the optimizer trajectory or independently reintegrate the ODE. The reader checks analytic forcing at its original tolerance and retains sealed source bytes for cancellation-sensitive residual subtraction; a prior cross-version rounding failure is preserved and no tolerance is widened. GPU timing and physical archives have fully verified cluster and local copies. Publication of compact result JSON files is not publication of every raw checkpoint or profiler trace.

The development, continuation and observation batches of Sections 6.7–6.9 each ran once on one four-GPU H100 node from a packet whose manifest lists every source hash. Every published source

is byte-identical to its executed copy, except the first continuation trainer, which is preserved only in that failed attempt's archive. The second continuation training attempt took 69 minutes. The four observation batches (the last running two frozen drivers in sequence) produced 72 observations with complete archives of 191, 581, 371, and 131 plus 161 members. All archives, including the failed first continuation attempt, are verified on cluster scratch and on persistent project storage; all owned GPU locks were released. The observation record `c36-observations-20261015.json` is generated from the cluster outputs by a published standard-library script; two further published scripts compute Fig. 4 and the rank correlations of Section 6.8 from that record. The baseline loss curves come from the continuation reader's per-run function applied unchanged to the development runs. The executed continuation reader labels its output with the development reader's state name; that inherited label is kept. A NumPy check of the exact width-w fields on the same holdouts supplies the field distances of Section 6.9.

The studies of Section 6.10 also ran from packets whose manifests list every source hash, and their published sources are byte-identical to the executed copies. The cancelled v4 attempt archived its partial outputs (157 members). The v5 training batch (446 members) used the unchanged development trainer and driver, so its in-batch reader labels its output with the development state name and applies the development branches; the registered v5 outcome comes from `decide_c36_v5`. Its observation batch archived 431 members. These archives are verified on cluster scratch and on persistent project storage. The v5 training lanes took 20,418–26,652 seconds against a 27,000-second guard, so no run was truncated.

The v6 batch of Section 6.11 ran once from the git archive of its pre-registration commit, so every executed source is byte-identical to the published one; the inputs are the published $\eta = 1/16$ data. Each lane spent about 8 minutes on start-up qualification, mostly the 131-point native subset. Training took 3,688–4,725 seconds against a 5,700-second guard, and each observation took about 30 minutes at chunk 512; the call-200 observations were skipped by the per-lane observation budget. The unchanged development reader reads the batch and labels it with the development state name, and the registered outcome comes from `decide_c36_v6`. The archive (396 members) is verified on cluster project storage; the login-node CPU self-test of the trainer was ended by the node's process limits, so the v6 additions were self-tested as components and the driver's control flow end to end.

The GH200 batches of Sections 6.12 and 6.13 ran on an ARM host with Python 3.13.5, Torch 2.14.1 (CUDA 13.0), Triton 3.8.0 and NumPy 2.5.4. They are not the H100 runtime, and no timing is compared across the two. Each batch was frozen from the committed files of its registration commit. The manifests list 113 and 133 files, and each driver verified every hash before starting. A small driver adapter disables the earlier per-process compiler-target shim, which the newer runtime does not need, through a frozen protocol field. The v7 batch (one node, two GPUs, 7,645 seconds) archived 246 members. Its in-batch reader output and decision are published byte-identical. The TTA cohort ran as five independent one-GPU jobs. Each archive (175–182 members) is sealed and read back member by member. The cohort record and the post-hoc rounding diagnosis are generated from those archives by two published scripts. The second cohort (Section 6.14) ran the same way from its own registration commit (141 files); its five archives hold 188–190 members, and the same publishing script generates its record. The paired cohort (Section 6.15) also ran from its registration commit (151 files), after a 16-minute qualification job on development seed 361901. Its five archives hold 233–237 members, and a published script generates its record. The first submission of its audits failed at argument parsing, before any check ran: the generated wrapper passed the three frozen hashes as one argument. That record is kept, and the audits were then run once with correct arguments. The frozen audits' failure records are kept unchanged. Full raw outputs stay on cluster project storage.

C.3 Ethier–Steinman campaign

The 18 cases ran as one batch on two GPUs of an existing allocation, holding the original per-GPU lane locks, while the node's other two GPUs carried another owner's workload. All 18 processes exit zero, and the locks are released before the CPU-only audit. The complete 126-member archive (19,218,796 bytes) is verified on cluster scratch, on persistent project storage and locally. The public source copies differ from the executed bytes only in a site activation path and a hardware label, and a release map lists both hashes [5]. The development history, including the nonconverged 100-block runs on one H20 and the CPU development run, is retained with the artifact.

D Every planned public solve

Table 35: All 54 planned public cells, without failed-attempt filtering. Setup plus optimization is in seconds. Closures are actual complete loss/gradient evaluations, including the 4,000 Adam evaluations; every run stops during L-BFGS. Six paired initialization seeds are reused across methods.

Seed	Method	Outcome	Setup + opt.	Closures
862101	FlashNS GL8	accepted	21.854	6833
862102	FlashNS GL8	accepted	32.786	9032
862103	FlashNS GL8	accepted	25.138	7279
862104	FlashNS GL8	accepted	21.068	6635
862105	FlashNS GL8	accepted	18.966	6261
862106	FlashNS GL8	accepted	30.989	8685
862101	Q10 compact	accepted	38.649	6733
862102	Q10 compact	accepted	53.417	9089
862103	Q10 compact	accepted	39.846	7076
862104	Q10 compact	accepted	35.679	6481
862105	Q10 compact	accepted	34.340	6309
862106	Q10 compact	accepted	45.944	8010
862101	Collapsed Taylor	accepted	41.665	7080
862102	Collapsed Taylor	accepted	48.582	8054
862103	Collapsed Taylor	accepted	39.165	6700
862104	Collapsed Taylor	accepted	38.941	6678
862105	Collapsed Taylor	accepted	35.904	6265
862106	Collapsed Taylor	accepted	53.893	8837
862101	Cao ED	accepted	53.866	6788
862102	Cao ED	accepted	66.355	8142
862103	Cao ED	accepted	55.754	6978
862104	Cao ED	accepted	53.004	6559
862105	Cao ED	accepted	49.436	6301
862106	Cao ED	accepted	65.182	8019
862101	Torch coordinate AD	accepted	65.903	6651
862102	Torch coordinate AD	accepted	94.703	9259
862103	Torch coordinate AD	accepted	72.703	7239
862104	Torch coordinate AD	accepted	68.661	6866
862105	Torch coordinate AD	accepted	61.316	6233
862106	Torch coordinate AD	accepted	84.040	8278
862101	STDE exact (2D)	accepted	77.324	6708
862102	STDE exact (2D)	accepted	92.869	7910

Seed	Method	Outcome	Setup + opt.	Closures
862103	STDE exact (2D)	accepted	79.977	6920
862104	STDE exact (2D)	accepted	76.264	6629
862105	STDE exact (2D)	accepted	73.246	6395
862106	STDE exact (2D)	accepted	101.451	8549
862101	PhysicsNeMo	accepted	73.710	6818
862102	PhysicsNeMo	accepted	93.615	8718
862103	PhysicsNeMo	accepted	60.251	6846
862104	PhysicsNeMo	accepted	70.725	6542
862105	PhysicsNeMo	accepted	71.302	6418
862106	PhysicsNeMo	accepted	95.941	8988
862101	cuDNN convolution AD	accepted	516.349	6709
862102	cuDNN convolution AD	accepted	699.179	9049
862103	cuDNN convolution AD	accepted	546.132	7110
862104	cuDNN convolution AD	accepted	498.826	6497
862105	cuDNN convolution AD	accepted	488.845	6367
862106	cuDNN convolution AD	accepted	662.827	8580
862101	cuEquivariance polynomial	accepted	35.119	6654
862102	cuEquivariance polynomial	accepted	45.720	7972
862103	cuEquivariance polynomial	accepted	36.338	6810
862104	cuEquivariance polynomial	accepted	34.258	6482
862105	cuEquivariance polynomial	accepted	33.449	6208
862106	cuEquivariance polynomial	accepted	47.182	8067